OPENSPEC / FIELD GUIDE

OpenSpec 中文完整使用教程

增订完整版 · 从第一次开发到团队交付

Todo 两轮完整实战 / 三人团队协作

完整源码 · 命令参考 · 流程与架构 · CI 与进阶

在原 34 页教程上扩充：每一步都说明输入位置、复制内容、结果检查与排错。

从初始化开发 Todo，到已有规格迭代，再到 Git 分支、PR 与规格冲突处理。

版本基线 v1.15.0 | 官方资料核对 2026-10-28

macOS / Linux / Windows

独立中文教程 · 88 页 · 完整源码与实测案例

从“已安装”走到“真正交付”

本书在原 34 页教程上扩充，原有主题均保留并重新编排。若第一次使用 OpenSpec，请先完成 Todo 第一轮，再学习第二轮已有规格迭代，最后阅读团队与命令参考。

你的目标	建议路线
安装与理解概念	核心概念 → 多平台独立安装 → 初始化与目录
初始化后不知道下一步	Todo List：从零到归档 → 步骤 01 至 12
修改已完成功能	优先级与截止日期七步迭代
与同事协作	三人分支 → PR → 合并 → 归档 → 规格冲突
查命令与进阶	CLI 速查 → 配置/Schema → Stores/Worksets → CI
直接运行参考项目	配套源码包 v1 或 v2；完整源码亦收录在书后

本次增订的交付与验证

配套项目零第三方依赖，Node ≥20.19.0 即可 npm start / npm test。v1 的 6 项和 v2 的 11 项模型测试已通过；两轮 delta 的严格校验、CLI 归档及主规格校验已实际执行。

浏览器实际检查了新增、完成、删除、刷新，以及同源旧数据升级后保留完成状态、修改优先级与日期、清空日期。AI 助手的自然语言输出因工具和模型而不同，书中报告是检查目标，不是保证逐字输出。

输入位置与记号

“终端”执行 openspec / Git / npm；“AI 聊天”输入 /opsx 或相应技能；“编辑器”保存文件；“浏览器”检验行为。DATE 表示真实归档日期，不能原样作为官方参数。书中团队与 CI 配置为教学建议。[R6][R12]

> 完成不是看到一个勾选列表：计划齐全、行为正确、测试有证据、主规格与已交付结果一致，才形成闭环。

目录 1/3

06 核心概念

07 多平台独立安装

08 包管理器与升级

09 初始化与目录

10 全流程图

11 计划产物与依赖

12 规格写法

13 默认六个 /opsx 工作流

14 扩展六个 /opsx 工作流

15 工具调用方式对照

16 Todo List：从零到归档

17 创建项目与初始化

18 初始化后的目录变化

19 三种助手如何使用

20 第一次分析需求

21 创建第一次提案

22 查看四类计划产物

23 Todo 第一轮完整提案

24 Todo 第一轮完整规格

25 Todo 第一轮设计与任务

26 审核与修改开发方案

27 Apply 开始真正开发

28 运行项目与功能测试

29 Verify 对照规格验证

30 Archive 完成归档

31 归档后的目录与下一步

32 优先级与截止日期：规划路线

33 分析与提出增量变更

目录 2/3

34 完整 Delta：替换与新增

35 评审、设计与任务

36 Apply 与旧数据迁移

37 测试、Verify 与归档

38 归档前后对照与提交

39 三名开发者与分支图

40 每个人的完整开发路径

41 Bob：支付功能完整操作

42 Chen：搜索功能完整操作

43 提交、Pull Request 与评审

44 合并后的规格归档

45 规格冲突：具体错误示例

46 冲突解决、验证与完成

47 需求与提案

48 可验收的 Delta 规格

49 设计、任务与最小代码

50 实现、测试与归档

51 修 Bug 与纯重构

52 设置与版本命令

53 查看、校验、归档

54 产物工作流命令

55 配置与 Schema 子命令

56 工具命令与旧入口

57 全部公开命令与别名

58 项目配置

59 自定义 Schema

60 分支、PR 与验收

61 Stores：跨仓库规划（Beta）

目录 3/3

62 References、Context 与 Worksets
63 架构与执行链路
64 环境与指令
65 规格、状态与归档
66 最佳实践与 CI
67 复杂功能拆分与依赖
68 可复制的 CI 配置
69 Todo 运行结果与文件说明
70 两轮项目启动配置 1/1
71 共用静态服务器 1/1
72 共用 HTML 页面 1/2
73 共用 HTML 页面 2/2
74 共用页面样式 1/1
75 共用浏览器交互 1/3
76 共用浏览器交互 2/3
77 共用浏览器交互 3/3
78 第一轮完整模型 1/2
79 第一轮完整模型 2/2
80 第一轮完整测试 1/1
81 第二轮完整模型 1/3
82 第二轮完整模型 2/3
83 第二轮完整模型 3/3
84 第二轮完整测试 1/2
85 第二轮完整测试 2/2
86 官方来源与版本说明
87 延伸阅读、术语与许可
88 上游 MIT License

OpenSpec 到底解决什么问题

只有聊天记录时，需求容易在多轮对话、换模型或换同事后丢失。OpenSpec 将计划写入仓库，让 AI 每次都能重新读取同一份目标与进度。它兼容新项目，也适合在已有代码库上做增量改动。[R1][R4]

概念	中文理解	主要位置
Spec	对当前系统行为的长期约定	openspec/specs/
Capability	一组内聚的系统能力，如 user-auth	specs/<能力路径>/spec.md
Change	一次有明确意图的变更	openspec/changes/<变更名>/
Delta spec	这次变更对规格的增量修改	changes/<变更名>/specs/
Artifact	计划产物：提案、规格、设计、任务等	变更文件夹内
Schema	产物、模板及其依赖关系的定义	内置或 openspec/schemas/
Profile	本机选择安装哪些助手工作流	全局 config.json
Archive	合并规格并保存变更历史	changes/archive/

两类入口，职责不同

终端里的 openspec init、validate、status 负责设置、结构检查与状态查询。AI 聊天里的 /opsx:propose、apply 等指令负责理解需求、编写文档或代码。CLI 不会因你运行 new change 就自动调用模型生成完整提案。

三个必须分清的“完成”

产物文件已存在，表示计划具备继续条件；tasks.md 勾选完，表示助手记录了执行进度；代码满足需求，需要测试和评审给出证据。三者应相互核对。OpenSpec 不替代编译器、测试框架、版本管理或发布系统。

> 判断一段内容应该放哪里：为什么改 → proposal；对外行为 → specs；如何实现 → design；具体执行与验证 → tasks。

首选：无需 Node.js 的安装方式

v1.15.0 提供独立可执行文件。安装脚本按系统和 CPU 架构下载发布资产，并比对 SHA256SUMS。你已安装时，先运行 openspec --version，无需重复安装。[R2][R3]

macOS / Linux：终端

```
curl -fsSL https://raw.githubusercontent.com/Fission-AI/OpenSpec/main/scripts/binary/install.sh | sh
openspec --version
```

默认位置是 ~/.local/bin。若安装提示 PATH 未包含此目录，按其提示加入后重新打开终端。Alpine 先安装运行库：apk add libstdc++ libgcc。

Windows：PowerShell

```
irm https://raw.githubusercontent.com/Fission-AI/OpenSpec/main/scripts/binary/install.ps1 | iex
openspec --version
```

默认目录为 %LOCALAPPDATA%\Programs\openspec，脚本将其加入用户 PATH。安装后旧终端找不到命令时，先新开一个 PowerShell 窗口。

平台与最低要求	手动下载资产
macOS 13+，Apple silicon / Intel	openspec-darwin-arm64 / openspec-darwin-x64
Linux glibc 2.17+，arm64 / x64	openspec-linux-arm64 / openspec-linux-x64
Alpine 3.17+，arm64 / x64	openspec-linux-arm64-musl / openspec-linux-x64-musl
Windows 10+，arm64 / x64	openspec-windows-arm64.exe / openspec-windows-x64.exe

手动安装：从官方 Releases 选对应文件，改名为 openspec（Windows 为 openspec.exe），放到 PATH 中。macOS/Linux 还需 chmod +x openspec。浏览器下载的 macOS 文件可能需要 xattr -d com.apple.quarantine openspec；只对已确认来源的下载操作。

> 脚本安装会执行远程代码。希望先审阅时，下载官方脚本检查后再运行；或采用手动下载与校验。可选构建来源验证：gh attestation verify <资产文件> --repo Fission-AI/OpenSpec。

沿用熟悉的包管理器也可以

Node 包安装方式要求 Node.js ≥ 20.19.0。Homebrew 与 Nix 会处理它们自己的依赖。Bun 全局安装 Node CLI 后仍需要系统中的 Node；现代 Yarn 不再提供 global add。[R3]

方式	安装命令	注意
npm	npm install -g @fission-ai/openspec@latest	先用 node --version 检查环境
pnpm	pnpm add -g @fission-ai/openspec@latest	需配置全局可执行目录
Yarn Classic 1.x	yarn global add @fission-ai/openspec@latest	Yarn 2+ 改用其他方案
Bun	bun add -g @fission-ai/openspec@latest	运行仍依赖 Node
Homebrew	brew install openspec	macOS/Linux；无需额外 tap
Nix	nix profile install github:Fission-AI/OpenSpec	项目提供官方 flake

Deno 属于进阶可选安装，需要授权读写、环境与网络等权限；使用官方安装页的完整命令，避免漏掉权限。一次性体验 Nix：nix run github:Fission-AI/OpenSpec -- --version。

升级分成两个层次

```
# 任选与你安装来源一致的一条
openspec install
npm install -g @fission-ai/openspec@latest
brew upgrade openspec

# 然后在每个项目中刷新助手指令
openspec update --dry-run
openspec update
```

openspec install 用于安装/升级独立版本；npm、Homebrew 用户执行它会切换安装来源。项目的生成指令不会随 CLI 自动刷新。update 在交互环境可提示新版本，但自动升级能力取决于安装来源。

避免多个副本抢占 PATH

macOS/Linux 用 which -a openspec，Windows 用 where.exe openspec。openspec version 可显示当前安装来源。nvm 切换 Node 后，原全局 npm 安装可能不再可见。

卸载前先 openspec completion uninstall，再使用对应包管理器卸载或删除独立文件。openspec/specs/ 和 archive/ 是项目知识资产，应按团队保留策略处理。

在项目根目录接入 OpenSpec

CLI 每台机器安装一次；init 则按项目运行。已有项目先确认当前路径与版本控制状态，再选择实际使用的助手，避免生成一堆无用集成文件。[R5][R8]

```
cd your-project
openspec init

# 自动化或明确知道工具时
openspec init --tools codex --language Chinese
# 同一项目同时支持两种工具
openspec init --tools claude,cursor
```

--language 用于新建项目配置；已有配置通过 context 调整语言。无交互运行请明确 --tools；--tools none 只设置配置，不安装助手集成。

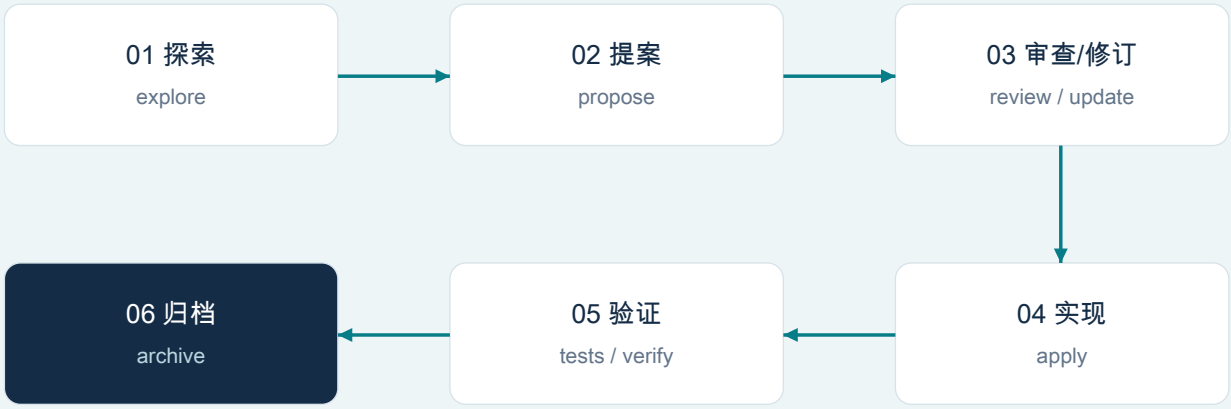
```
your-project/
  src/                          业务代码
  openspec/
    config.yaml                   项目上下文与规则
    specs/                      当前系统规格，按需出现
    changes/                    活跃变更，按需出现
      add-dark-mode/
        .openspec.yaml        变更元数据
        proposal.md
        specs/theme/spec.md
        design.md             按需要生成
        tasks.md
      archive/                已完成变更，按需出现
  .agents/skills/openspec-*/        选择 Codex 后的集成
```

第一次初始化后的检查

1. 确认 openspec/config.yaml 存在；空项目中的 specs/、changes/ 可以尚未出现。
2. 看 init 结尾的 Getting started 提示，使用它给出的准确调用方式。
3. 重启或重新加载 AI 工具，确认技能/命令可见。
4. 将项目上下文、能力规格和变更纳入版本管理。生成集成文件的提交策略由团队统一。

> 不要把个性化规则直接改进生成的 SKILL.md。它们可能被 update 覆盖；团队约束放 config.yaml、AGENTS.md 或项目文档。

一条从想法到归档的闭环



发现新信息时回到计划修订；归档后进入下一次变更。

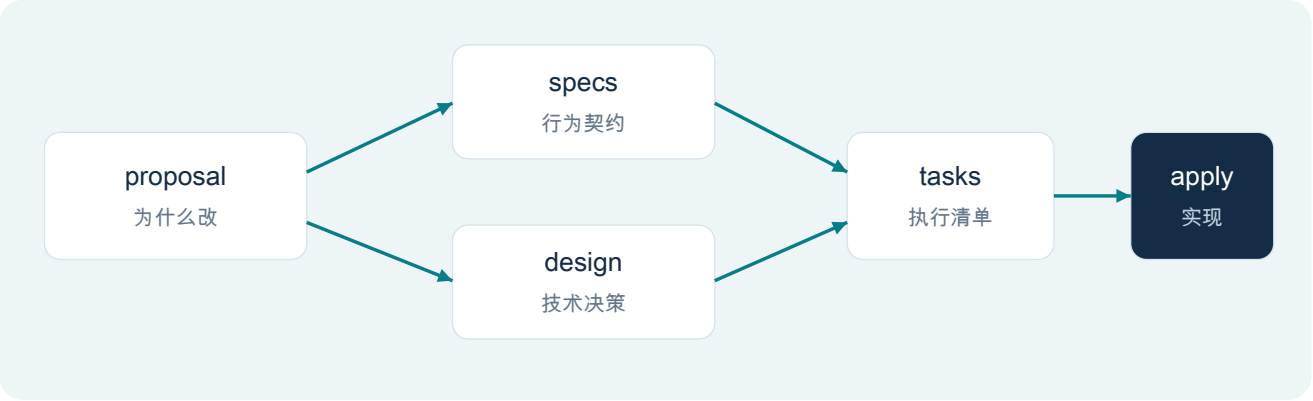
每一步你要判断什么

步骤	应得到什么	人的检查点
Explore	问题、边界、候选方案	是否已有能力？需求是否明确？
Propose	可审查的变更计划	规格是否定义了真正可接受的结果？
Review / Update	一致的提案、规格、设计和任务	是否遗漏异常、兼容和迁移？
Apply	代码、测试、任务进度	每项完成是否有实际证据？
Verify	结构检查、行为测试、评审意见	是否满足需求与设计？
Archive	当前主规格 + 带日期的历史变更	同步内容是否真实反映交付？

可在任意阶段回到计划修订。需求已经明确时可以从 propose 开始。verify 为可选助手工作流，测试与评审仍应按项目需要执行。[R4][R6]

> 归档不是发布，也不会替你合并 PR。Git、CI 与部署由已有工程流程负责。

四份文档各司其职



proposal 是起点；specs 和 design 都依赖 proposal；tasks 汇合它们；apply 读取 tasks。图表示默认 schema 的依赖，两个就绪产物不必同时生成。[R9]

文件	写什么	审查问题
proposal.md	Why、What Changes、Capabilities、Impact	意图单一吗？能力路径准确吗？
specs/<能力>/spec.md	行为增量、需求、场景	测试人员能据此判定通过吗？
design.md	决策、备选方案、风险、迁移	方案与代码库约束一致吗？
tasks.md	小而可验证的复选框任务	规格中的行为是否都有任务覆盖？

“可省略”需要依据

简单变更没有架构、依赖、数据模型等设计决策时，默认流程可按条件省略 design。纯重构、工具或文档变更如果没有行为变化，可在 .openspec.yaml 声明 skip_specs: true，明确省略 specs。不要为了凑齐文件虚构需求。

文件进度不等于质量认证

status 根据实际文件与 schema 推导 ready、blocked、done 等状态；它不会理解一篇规格是否正确。JSON 中 isPlanningComplete 表示计划产物齐备，isComplete 是兼容别名，二者都不代表任务已实现。任务完成应查看 tasks 或 instructions apply 的任务状态。

何时拆分变更

一个变更应该能用一句话说明意图。登录改造、报表导出与主题切换通常应拆开；某个需求横跨前后端但必须一起验收时，可以保留为同一变更并明确接口边界。

将“好用”改写为可验证的行为

保留 OpenSpec 识别的英文标题与关键字，标题后的名称和正文可以用中文。Requirement 是需求，Scenario 是验证该需求的具体情境；场景标题必须是四级标题。[R9][R10]

```
## ADDED Requirements

### Requirement: 保存主题偏好
系统 SHALL 在浏览器存储可用时保留用户选择的主题。

#### Scenario: 刷新后保留深色
- **GIVEN** 用户已选择深色且存储成功
- **WHEN** 用户刷新页面
- **THEN** 页面仍显示深色
```

Delta 标题	用途	最容易犯的错
ADDED Requirements	新需求	把已有需求的修改当成新增
MODIFIED Requirements	替换既有需求	只写修改的一句话，丢失旧场景
REMOVED Requirements	删除需求	未写 Reason 与 Migration
RENAMED Requirements	仅重命名	旧名不匹配或同时改了行为

```
## RENAMED Requirements
- FROM: `### Requirement: Theme selection`
- TO: `### Requirement: 主题选择`
```

MODIFIED 应复制现有需求的完整块，包括全部场景，再编辑；名称要与主规格对应。重命名且修改行为时，需要配合 MODIFIED，而不能只使用 RENAMED。

写强规格的四个检查

1. 用 SHALL / MUST 表达必须满足的行为，避免“尽量”“合理”等模糊措辞。
2. 给出输入、输出、错误和边界条件，不把内部类名或框架选择写成外部需求。
3. 每个需求至少有一个真正检验它的场景；重要需求同时覆盖异常路径。
4. 新能力写清 Purpose，避免归档后留下 TBD。Purpose 建议至少 50 字符，需求描述尽量不超过 500 字符，详细例子放场景。

在 AI 聊天中输入这些指令

本页使用官方通用写法。Codex 等工具请换成“工具调用方式对照”中的技能名。终端无法直接执行 /opsx:propose。[R6][R8]

指令	中文解释	示例
/opsx:explore [主题]	读取项目、调查问题、比较方案；不写代码。默认不写文件，明确要求记录时可写计划	/opsx:explore 深色模式如何接入
/opsx:propose [名称或描述]	建立变更并按依赖生成计划；到可审查阶段停止	/opsx:propose add-dark-mode
/opsx:apply [变更名]	读取计划并实施未完成任务，记录复选框进度；遇阻塞暂停	/opsx:apply add-dark-mode
/opsx:update [变更名]	修改已有计划，协调文档一致性；逐项确认修改，不改代码	/opsx:update add-dark-mode
/opsx:sync [变更名]	将 delta 合并到主规格；变更仍保持活跃	/opsx:sync add-dark-mode
/opsx:archive [变更名]	检查完成度、处理待同步规格并将变更移到历史目录	/opsx:archive add-dark-mode

一次典型的需求调整

```
/opsx:update add-dark-mode
新增约束：浏览器禁用持久化存储时仍应允许切换，
但允许刷新后丢失选择。请同步检查规格、设计和任务。
```

批准修订后再 /opsx:apply add-dark-mode，使代码跟上新计划。update 主要维护已存在的产物；完全未开始的产物由 continue 或 CLI instructions 引导生成。

两种 update、两种 archive

openspec update 刷新工具集成；/opsx:update 修订变更计划。

openspec archive 是终端中的解析、校验与归档命令；/opsx:archive 是助手流程，包含解释、检查和语义同步交互。不要将它们当成相同入口的不同拼写。

> 有多个活跃变更时显式写名称。归档前确认未完成事项与同步目标，不把助手的“已完成”当成测试结果。

需要更细控制时再启用

先在终端运行 openspec config profile，在选择器中添加所需工作流，再在项目内 openspec update。默认 core 并不安装下表的六项。[R6][R7]

指令	中文解释	适合何时
/opsx:new [名称]	只建立变更骨架与元数据，展示第一份模板，不生成完整计划	复杂需求需要逐份审查
/opsx:continue [名称]	按依赖创建下一份就绪产物，一次一份	每一步都要控制细节
/opsx:ff [名称或描述]	按依赖补齐实现所需的计划产物；不写业务代码	需求明确、希望快速规划
/opsx:verify [名称]	只读核对实现与计划的完整性、正确性、一致性并给报告	归档前检查遗漏与偏离
/opsx:bulk-archive	选择多个变更，检查同步冲突并批量归档	一批并行工作已经结束
/opsx:onboard	用实际代码库走一遍变更到归档的教学流程，会产生真实文件	希望边做边学

两条计划路线

```
快速规划：propose → 审查 → apply
逐份规划：new → continue → continue → … → 审查 → apply
```

ff 与 propose 都有快速生成计划的用途。新手用 propose 即可；启用逐份路线后，按已安装版本的提示使用 continue 或 ff。已有变更优先 continue/update，避免重复建立目录。

看懂 verify 报告

Completeness 检查任务、需求和场景是否覆盖；Correctness 检查实际行为是否符合意图；Coherence 检查设计与实现是否一致。CRITICAL 优先解决，WARNING 需要判断和解释，SUGGESTION 是改进建议。

> verify 为只读报告，不自动修复、不运行发布、不保证归档被阻止。需要硬性门禁时，在评审规则和 CI 中明确执行。

同一个工作流，不同工具的拼写

以 init 最后输出的提示与工具中的技能选择器为准。下面给出常见集成，完整工具列表见参考资料。[R8]

工具	创建提案	实施变更
Claude Code / Gemini CLI	/opsx:propose	/opsx:apply
Cursor / Copilot IDE	/opsx-propose	/opsx-apply
Antigravity / OpenCode	/opsx-propose	/opsx-apply
Amazon Q	@opsx-propose	@opsx-apply
Codex	$openspec-propose	$openspec-apply-change
Kimi Code	/skill:openspec-propose	/skill:openspec-apply-change
仅技能的兼容工具	/openspec-propose	/openspec-apply-change

Codex 中十二项工作流的准确技能名

通用工作流	Codex 调用	通用工作流	Codex 调用
explore	$openspec-explore	propose	$openspec-propose
apply	$openspec-apply-change	update	$openspec-update-change
sync	$openspec-sync-specs	archive	$openspec-archive-change
new	$openspec-new-change	continue	$openspec-continue-change
ff	$openspec-ff-change	verify	$openspec-verify-change
bulk-archive	$openspec-bulk-archive-change	onboard	$openspec-onboard

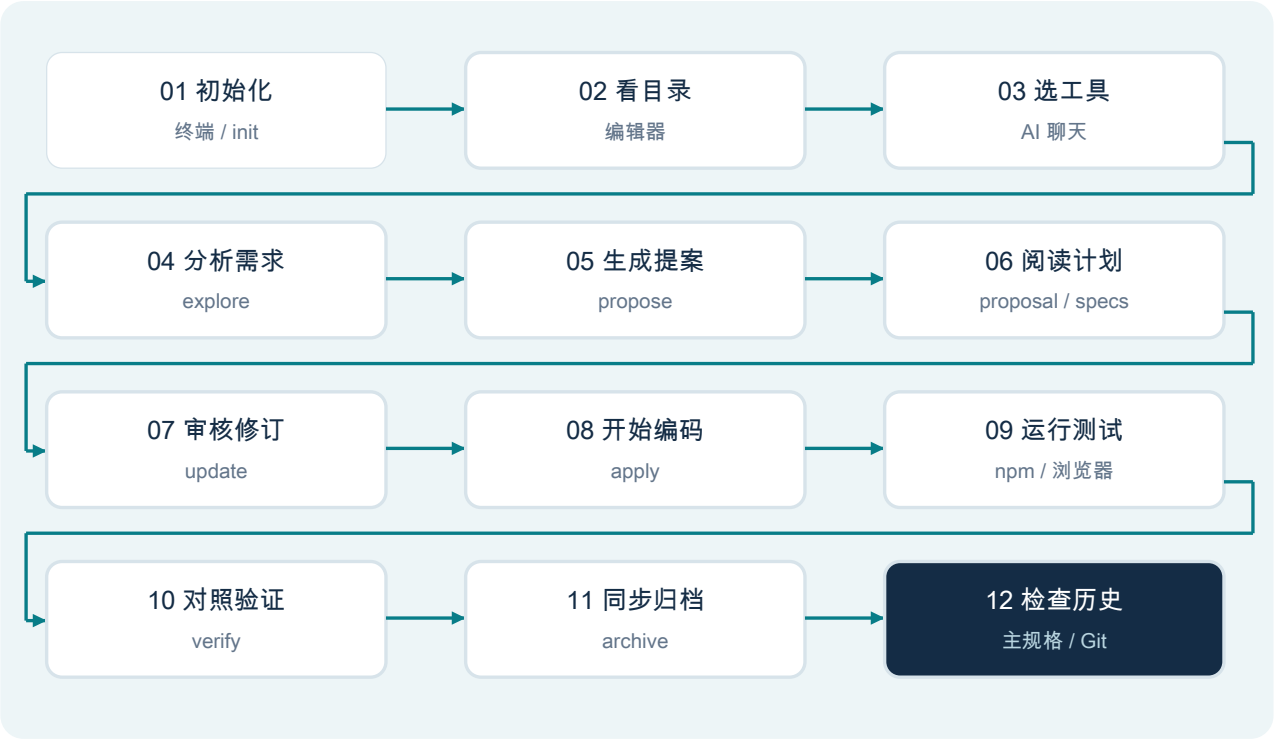
Codex 使用 .agents/skills/openspec-*/SKILL.md；即使 delivery 为 commands，Codex 集成仍使用 skills。不要照搬旧教程去找全局 ~/.codex/prompts/opsx-*.md。

工具边界

Copilot IDE 的 prompt 文件与 Copilot CLI、云端 coding agent 是不同集成。云端配置需单独选择，会生成 GitHub Actions 等文件。某些纯技能工具依靠自然语言匹配技能，并非都把 /openspec-* 注册为可输入命令；找不到时明确要求“使用 openspec-propose 技能”。

首次开发完整实战

本章按执行顺序开发一个可在浏览器使用的 Todo List。第一轮实现新增、完成/恢复、删除和本地保存；第二轮在同一能力规格上增加优先级与截止日期。原有深色模式案例保留在后续，供对照不同场景。



准备条件与交付形态

Node.js ≥20.19.0 用于本地服务器与测试；OpenSpec CLI 使用核对版本 v1.15.0；准备 Cursor、Claude Code 或 Codex 中任意一个已能使用的 AI 编码助手。OpenSpec 本身不会替你开通模型服务。

配套源码包有 v1/ 和 v2/ 两份完整项目。想练习 AI 流程时，从空文件夹跟随步骤，不要先复制完成版；需要对照或排错时再查看源码。两版共用存储键和浏览器地址，便于体验升级保留旧数据。

技术选择	用途与取舍
原生 HTML/CSS/JS	教学项目无需框架构建；大型产品应沿用已有工程栈
localStorage	演示刷新持久化，单浏览器保存；不提供账号或云同步
Node 内置 HTTP / node:test	无第三方依赖，启动和测试命令固定
OpenSpec + Git	文件记录计划与历史，Git 管理分支、评审和回退

> 全章 /opsx:… 是通用名称。Cursor 改为 /opsx-…；Codex 改为 $openspec-… 的准确技能名。下一步的工具表给出可直接复制的写法。[R6][R8]

01 创建或打开项目

操作目的与输入位置

在正确的项目根目录安装 OpenSpec 集成。下面全部输入终端；macOS/Linux 的 shell 和 Windows PowerShell 都能逐行执行这些基础命令。已有项目直接 cd 到项目根目录，跳过 mkdir。

```
node --version
openspec --version
mkdir todo-list
cd todo-list
git init
openspec init --tools cursor,claude,codex --language Chinese
```

仅用一种助手时可改为 --tools cursor、--tools claude 或 --tools codex。无需把三种工具全部安装。Git 是为后续版本历史准备，与 OpenSpec CLI 安装独立。

预期文件与结果

项目内出现 openspec/config.yaml，以及所选工具的技能/命令文件。此时还没有 Todo 代码、package.json 或可运行网页；init 不会自动生成业务应用。

如何检查成功

在编辑器中打开 todo-list，找到 config.yaml，确认 schema: spec-driven。终端执行以下命令，确认 CLI 和当前目录可识别：

```
openspec list
openspec config list
```

空项目没有活跃 change 是正常结果。config list 展示本机偏好，不是 Todo 需求文档。

常见错误与解决

命令不存在：检查安装与 PATH，重新打开终端。Node 过旧：更新 Node 后再运行案例。目录错误：先检查终端当前路径，再 init；误生成到别处应先核对文件后清理。Git 要求身份信息时，按自己的团队身份配置，不要复制他人的邮箱。

> 不要先运行 npm start：应用文件尚未产生。真正开始写代码在步骤 08，启动在步骤 09。

02 读懂初始化生成的文件

操作目的与输入位置

在编辑器文件树查看初始化结果，区分工具配置、当前规格与变更计划。终端可以重新运行只读命令核对；不要把树形示意图当命令执行。

```
todo-list/
  openspec/
    config.yaml
  .cursor/
    skills/openspec-*/SKILL.md
    commands/opsx-*.md
  .claude/
    skills/openspec-*/SKILL.md
    commands/opsx/*.md
  .agents/
    skills/openspec-*/SKILL.md
```

上图是 delivery=both 时选择三种工具的典型结果；未选的工具不出现，Codex 只有技能。specs/、changes/ 可在首次产生内容时再建立，不要因空目录暂缺就认定失败。

可复制操作：补充项目约束

用编辑器调整 openspec/config.yaml，保留已有配置，仅合并以下内容：

```
schema: spec-driven
context: |
  这是一个原生 HTML/CSS/JavaScript Todo 教学项目。
  使用 Node 内置服务器与 node:test，不引入第三方依赖。
  中文编写正文，保留 OpenSpec 英文结构标题。
  第一轮只实现标题、完成状态、删除和本地持久化。
```

预期结果与成功检查

保存后文件是合法 YAML；下一次 instructions 会读取新上下文。后续 propose 的方案应遵循无依赖技术选择，并把优先级和截止日期留到第二轮。

常见错误与解决

修改生成的 SKILL.md 后被覆盖：把项目规则移回 config.yaml。配置文件同时有重复 schema/context 键：合并成单个键。使用 Tab 缩进：改为空格。此时无 change，不能靠 instructions --change add-todo-list 检查，因为该变更尚未创建。

03 在正确的聊天框发出指令

操作目的与输入位置

打开 todo-list 项目或允许助手访问该目录，重载技能后，在 AI 聊天框输入。下面每格为对应工具的准确入口，选择自己工具的一列，不要混用。

动作	Claude Code	Cursor	Codex
分析	/opsx:explore	/opsx-explore	$openspec-explore
提案	/opsx:propose	/opsx-propose	$openspec-propose
修订	/opsx:update	/opsx-update	$openspec-update-change
实现	/opsx:apply	/opsx-apply	$openspec-apply-change
验证	/opsx:verify	/opsx-verify	$openspec-verify-change
归档	/opsx:archive	/opsx-archive	$openspec-archive-change

可复制操作：先启用 verify

以下输入终端；选择器中保留 core 六项，并额外勾选 verify，然后在当前项目刷新：

```
openspec config profile
openspec update
```

再次重载助手。verify 属于扩展工作流，默认 core 没有安装它；不要虚构 openspec verify 这样的终端命令。[R6][R7]

预期结果与成功检查

Claude Code 在项目会话中能识别 /opsx:explore；Cursor 的 Agent 聊天能选中 /opsx-explore；Codex 能调用项目内的 openspec-explore 技能。init 的结尾提示优先于旧教程截图。

常见错误与解决

输入终端后“命令不存在”：移到助手聊天。扩展命令仍缺失：确认 profile 已保存、在正确项目执行 update，并重开会话。工具只提供技能入口：通过其技能选择器，或明确说“使用 openspec-explore 技能分析当前项目”。本机全局设置不会自动刷新其他项目。

04 用 explore 明确要做什么

操作目的与输入位置

在 AI 聊天框分析范围、现有代码和验收边界，暂不生成业务代码。下面以 Claude 通用写法为例，Cursor/Codex 替换首行即可。

```
/opsx:explore
我要从这个空项目开发 Todo List。
第一轮：新增任务、标记完成与恢复、删除任务、刷新保留。
标题去掉首尾空格后必须为 1 至 120 个字符。
只用原生 HTML/CSS/JS、localStorage、Node 内置服务器和测试。
请确认空列表、空标题、损坏存储、存储写入失败的行为。
优先级、截止日期、账号与云同步先不做。
先给需求清单、风险和技术选择，不写业务代码。
```

预期文件与结果

默认只得到调查和讨论，不产生 Todo 业务文件。输出应明确：空标题拒绝且提示；空列表有提示；非法存储读入为空列表；写入失败仍支持本会话操作并提示刷新可能丢失。

如何检查成功

用自己的话复述“第一轮完成是什么”。确认按钮可键盘操作，任务标题以普通文本显示，存储失败不会中断页面。若助手建议 React、数据库或登录，要求回到本轮已确认范围。

常见错误与解决

助手开始编码：要求停在需求分析并回顾范围；尚未审查的代码不应当作既定方案。漏掉异常路径：补问“浏览器禁止存储时如何表现”。讨论很长但无法推进：将待确认项收敛到会影响规格的决策，再进入 propose。

> explore 默认不写文件；明确要求记录结论后可写计划，但它不会进入业务实现。需要快速跳过分析时，前提是需求和边界已经足够明确。

05 用 propose 生成可审查计划

操作目的与输入位置

在 AI 聊天框把确认过的需求写入 change 文件夹。目录名用 add-todo-list，能力名用 todo-items，前者表示这次工作，后者表示长期行为。

```
/opsx:propose add-todo-list
按刚才确认的第一轮 Todo 范围生成计划。
能力名使用 todo-items。保留英文结构标题，正文用中文。
生成 proposal、delta specs、design 和带验证方法的 tasks。
设计说明 localStorage 失效降级，以及无依赖启动和测试。
所有代码尚未开始；先停下来让我审查方案。
```

预期文件与结果

```
openspec/changes/add-todo-list/
  .openspec.yaml
  proposal.md
  specs/todo-items/spec.md
  design.md
  tasks.md
```

本例有持久化与异常处理决策，所以保留 design。提案明确为什么做、能力边界和影响，尚不产生可运行 Todo 网页。

如何检查成功：终端

```
openspec list
openspec status --change add-todo-list
openspec show add-todo-list --type change
```

应看到这个活跃变更及产物状态。status 的 done 只表示产物存在，不代表产品已开发。

常见错误与解决

名称重复：先 list，继续现有 change，不要为了重跑创建 add-todo-list-2。计划出现优先级：要求移到未来变更。只有空目录：确认调用的是 propose，而非 CLI new change 或可选 new 骨架流程。

06 逐份理解 AI 打算如何开发

操作目的与输入位置

在编辑器打开变更目录阅读 Markdown；终端核对状态与结构；在 AI 聊天中询问不清楚的设计。按 proposal → specs → design → tasks 顺序阅读。[R9]

产物	本例应包含的内容	不应混入
proposal	单用户 Todo 第一轮、todo-items 能力、页面与测试影响	账号、支付、云同步
specs	增删/完成、输入校验、持久化与失败场景	内部函数名替代行为
design	原生页面、模型与 UI 分离、同一存储键、安全文本渲染	只有“按最佳实践实现”
tasks	模型/测试、页面交互、存储异常、运行与验收	不可验证的“完善功能”

可复制检查：终端

```
openspec status --change add-todo-list --json
openspec validate add-todo-list --strict --no-interactive
openspec instructions apply --change add-todo-list --json
```

预期文件与结果

上述命令只读取/校验，业务代码仍未创建。validate 应通过；apply 的 instructions 应列出待完成任务。正常生成的任务是 - [ ]，不是一开始全部 - [x]。

如何检查成功

每条重要需求能在 tasks 找到执行与验证方法；失败路径能在 spec 找到场景；设计说明如何落实它。下一节给出本例完整的规范参考，可与 AI 的实际产物对照。

常见错误与解决

validate 通过却内容明显不全：它不能替代人工语义审查。instructions 报缺依赖：先查 status 的 missingDeps，补齐产物。看见空 design：要求助手解释条件省略还是漏写，不要为了状态好看保留空文件。

proposal.md：明确单一意图

下面是配套案例中的完整提案正文。将其与自己的方案比较，而不是要求每次生成逐字相同。能力路径与下页 delta 一致。

```
## Why
用户需要一份能在刷新后保留的个人待办清单，避免把任务只放在临时页面状态中。

## What Changes
- 支持新增、完成与恢复、删除和空列表提示。
- 校验标题；本地保存；存储异常时保持本会话可用。
- 优先级、截止日期、账号与云同步不在本轮范围。

## Capabilities
### New Capabilities
- todo-items: 单浏览器内的待办事项生命周期与本地保存。
### Modified Capabilities
无。

## Impact
新增网页、模型、原生交互、Node 本地服务器与内置测试；不引入第三方依赖和后端数据库。
```

本轮为什么不把后续功能塞进来

让“新增、完成、删除、刷新保留”先形成一个可运行闭环，用户可以立即验证。优先级和截止日期需要数据兼容约定，在第二轮对同一能力增加增量需求；这能直接展示已有规格如何演进。

审查提示词：AI 聊天

```
请按 proposal 中的范围逐项核对 specs、design 和 tasks。
标出未覆盖需求、过度设计和互相矛盾的描述。
先给审查报告，不修改代码，也不要直接全部勾选任务。
```

specs/todo-items/spec.md

以下是第一轮完整 delta，结构关键字保持英文。归档后 ADDED 块会成为主规格 Requirements 中的需求，后续应在主规格上演进。

```
## Purpose
本能力定义单浏览器中的待办事项管理行为，包括任务新增、完成与恢复、删除、输入校验及本地保存，以保证用户在刷新页面和存储异常情况下能理解任务状态与数据边界。

## ADDED Requirements

### Requirement: 任务操作
系统 SHALL 支持新增任务、切换完成状态和删除指定任务。
#### Scenario: 新增与完成
- **WHEN** 用户添加有效标题并勾选该任务
- **THEN** 列表显示任务且标记为完成，取消勾选后恢复未完成
#### Scenario: 删除指定任务
- **WHEN** 用户删除两个任务中的一项
- **THEN** 仅移除选定任务，其他任务保持原状
#### Scenario: 空列表
- **WHEN** 当前没有任何任务
- **THEN** 页面显示空列表提示

### Requirement: 标题校验
系统 SHALL 去掉标题首尾空格，拒绝空白及超过 120 个字符的标题。
#### Scenario: 非法标题
- **WHEN** 用户提交全空白或超过 120 个字符的标题
- **THEN** 显示校验提示且不新增任务
#### Scenario: 普通文本
- **WHEN** 标题包含 HTML 标记文本
- **THEN** 按文字显示而不作为可执行标记插入页面

### Requirement: 保存任务
系统 SHALL 在存储可用时保留任务，存储写入失败时继续支持本会话操作。
#### Scenario: 刷新恢复
- **WHEN** 用户添加或修改任务后刷新同一浏览器源的页面
- **THEN** 成功保存的任务 ID、标题与完成状态保持
#### Scenario: 存储失败
- **WHEN** 存储读写不可用或内容损坏
- **THEN** 页面不崩溃，无法读取时使用空列表，无法写入时提示刷新可能丢失修改
```

design.md 与 tasks.md

这两份文档把行为落到技术选择和执行顺序。模型与 UI 分离是本例的设计决策，不是 OpenSpec 对所有项目的强制架构。

```
## Context
见 proposal.md；这是无框架教学网页，需要跨刷新保存和可解释的失败降级。
## Decisions
- 原生 HTML/CSS/JS：没有依赖安装和构建步骤；复杂产品应沿用现有框架。
- 模型与 UI 分离：模型可用 node:test 检查，页面操作另做浏览器验收。
- localStorage：同源持久化；不可用时保留内存状态并提示。
- DOM textContent 渲染任务标题，避免把标题当作标记执行。
## Risks / Trade-offs
本例单浏览器、单标签页，无云同步和并发协调；存储失败后刷新会丢失未保存操作。
```

```
## 1. 模型与验证
- [ ] 1.1 实现任务创建、完成与输入校验，运行模型测试通过
- [ ] 1.2 实现读取、保存与失败降级，测试存储往返和异常
## 2. 页面与验收
- [ ] 2.1 实现新增、完成、删除与空态，浏览器逐项检查
- [ ] 2.2 使用普通文本渲染标题，检查键盘操作与刷新保存
- [ ] 2.3 提供无依赖启动脚本与说明，运行 npm test 通过
## Workflow follow-up
- 审查通过后归档，检查主规格与历史目录。
```

07 先修正计划，再开始编码

操作目的与输入位置

在 AI 聊天框用 update 修订已经存在的计划。假设 AI 把存储失败写成“禁用全部按钮”，这与我们希望本会话继续操作的目标冲突。

```
/opsx:update add-todo-list
请把存储失败策略改为：界面继续支持新增、完成和删除，
显示“无法保存，刷新可能丢失修改”；不禁用全部操作。
检查 proposal、specs、design、tasks 是否有矛盾，
列出具体修订并按工作流逐份让我确认。不要修改业务代码。
```

预期文件与结果

已有计划文件按确认内容修订；补齐异常场景、实现策略和验证任务。若 AI 输出的原方案已经正确，本步可只做一致性检查，不需要人为制造修改。

如何检查成功：终端

```
openspec validate add-todo-list --strict --no-interactive
git diff -- openspec
```

尚未提交的全新文件可能不会出现在普通 git diff 中，用 git status --short 和编辑器检查未跟踪文件。确认规格仍保留新增、完成、删除、刷新持久化等全部原需求。

常见错误与解决

只改 design 却漏改 spec：要求做产物一致性检查。完全缺少某一产物：用可选 continue，或按 status/instructions 补齐；update 不负责从零开始所有产物。把 openspec update 输入终端以为会改需求：它只刷新集成，应使用本页助手指令。

> 评审通过的信号是“范围、行为、方案、任务一致”。没有必要追求与书中完全相同的措辞，但不能省略关键验收。

08 让 AI 按任务实现应用

操作目的与输入位置

新开干净的项目 AI 会话，显式指定 add-todo-list。在聊天框中启动实现，助手应先读取当前文件，不依赖你手工重述全部讨论。

```
/opsx:apply add-todo-list
请先读取变更产物与项目约束，再按 tasks 逐项实现。
生成 package.json、server.mjs、index.html、style.css、
model.mjs、app.mjs、model.test.mjs。
提供 npm start 与 npm test；不添加第三方依赖。
标题只按文本渲染。每完成一项验证后再勾选。
遇到规格不明确或测试失败先说明并处理，不跳过。
```

预期文件与结果

项目根目录出现可运行网页和测试。tasks.md 从未完成逐步变为已完成。完整参考代码在本书“源码附录”；配套包 v1 是可运行的第一阶段结果。

如何检查成功：终端

```
git status --short
npm test
openspec status --change add-todo-list
```

检查文件是否齐全，并看真实测试输出。任务勾选不代表功能自动正确，下一步还要启动页面并手工检查。

常见错误与解决

助手引入框架或数据库：指出已确认约束并审查额外文件。突然停止：新会话继续相同 apply，依据文件和未完成任务恢复。模型只给代码块不保存：明确要求写入当前项目。源码参考中的文件名与 AI 生成不同：以实际 package.json 启动命令为准，或让助手统一到本书布局。

09 第一次看到可用的 Todo 页面

操作目的与输入位置

在终端运行测试；再启动开发服务器并保持终端打开。在浏览器访问输出地址，逐项操作页面。以下是本书配套代码的真实命令：

```
npm test
npm start
```

浏览器打开 http://127.0.0.1:5173。无需 npm install：本例没有第三方依赖；Node/npm 已安装即可。停止服务器用 Ctrl+C。

页面操作	预期结果与成功标准
空白或全空格标题 → 添加	出现校验提示，列表不新增
输入“ 学习 OpenSpec ” → 添加	新条目标题为“学习 OpenSpec”
勾选 → 取消勾选	完成态与未完成态可逆
添加第二项 → 删除第一项	第二项保持，空列表有友好提示
添加任务后刷新同一地址	标题、ID 与完成状态保留
输入带 HTML 的标题	当普通文字显示，不插入可执行标记
键盘 Tab / Space / Enter	控件可聚焦、按钮和复选框可操作

预期文件与检查证据

测试只输出结果，不生成生产构建；node:test 是业务测试，不是 OpenSpec 命令。浏览器数据保存在当前源 localStorage。保存测试结果和关键页面观察，供 verify 参考。

常见错误与解决

Missing script: start：检查所在目录与 package.json。EADDRINUSE：停止占用 5173 的旧示例，或换端口。改成 5174、localhost 或不同浏览器后数据“丢失”：存储按源隔离，应回到同一地址。不要双击 index.html 用 file:// 打开 ES 模块页面。

10 用证据核对代码与计划

操作目的与输入位置

先在终端执行结构与业务校验，再在 AI 聊天运行可选 verify。verify 输出只读报告，不等于自动执行了全部测试。

```
openspec validate add-todo-list --strict --no-interactive
npm test
```

```
/opsx:verify add-todo-list
请对照每条需求和场景检查代码及测试证据，
分别报告完整性、正确性和一致性。
明确区分已执行的自动测试、已观察的页面行为和未验证项。
特别核对存储异常、空标题和删除其他任务不受影响。
只给报告，不修改文件、不归档。
```

预期文件与结果

报告包含 Completeness、Correctness、Coherence，问题按 CRITICAL、WARNING、SUGGESTION 分类。本书固定参考代码第一阶段有 6 项模型测试；UI 的新增/切换/删除还需浏览器验证。你的 AI 实现可能有不同数量的测试。

如何检查成功

每项结论能指向需求、文件或测试结果。遇到真正的问题，用 apply 修复代码，或在需求确实改变时 update 计划，再重跑相关测试；不能为了让报告好看而降低已承诺行为。

常见错误与解决

指令不存在：回步骤 03 启用 verify。出现 openspec verify：这是不存在的终端入口，应改用助手指令。助手把 status done 当验收：要求重新核对代码与实际测试。报告说“应该通过”但没有执行记录：标记未验证，再实际运行。

11 把已确认的行为写入主规格

操作目的与输入位置

代码、规格和测试都已检查后，在 AI 聊天请求归档。单人案例可在同一分支完成；团队按后文统一 PR 与归档时机。

```
/opsx:archive add-todo-list
请检查产物和任务完成度，将本次 delta 同步到主规格，
确认同步摘要后归档；汇报主规格路径和历史目录。
如果有未完成任务或冲突，请先说明，不静默跳过。
```

预期文件与结果

新增 openspec/specs/todo-items/spec.md，活跃目录移动到 changes/archive/YYYY-MM-DD-add-todo-list/。所有提案、设计、任务和 delta 保留在历史中。日期由实际归档日生成，书中 DATE 仅代表这个日期。

如何检查成功：终端

```
openspec list
openspec list --specs
openspec show todo-items --type spec
openspec validate --all --strict --no-interactive
openspec validate --archived --no-interactive
```

活跃变更不再列出 add-todo-list，能力列表包含 todo-items，主规格描述已完成的 Todo 行为。仅“目录被移动”而没有正确主规格不算成功。

常见错误与解决

选择跳过同步后主规格未更新：按归档摘要检查并修复，避免再次盲目重复合并。非交互脚本无法确认：在已审查目标后可用 openspec archive add-todo-list --yes。该命令确实改文件；instructions archive 仅取输入，不执行归档。[R12]

12 确认项目完成第一轮闭环

操作目的与输入位置

在编辑器查看归档后的目录，终端做最终检查并用 Git 保存。以下是结构示意，日期以真实操作为准。

```
todo-list/
  package.json / server.mjs / index.html / style.css
  model.mjs / app.mjs / model.test.mjs
  openspec/
    config.yaml
    specs/todo-items/spec.md          当前长期规格
    changes/archive/
      DATE-add-todo-list/
        .openspec.yaml
        proposal.md
        specs/todo-items/spec.md  第一轮 delta 历史
        design.md
        tasks.md
```

可复制检查与保存：终端

```
npm test
openspec validate --all --strict --no-interactive
git status --short
git add .
git commit -m "feat: add todo list with archived spec"
```

检查暂存内容后再提交；仓库内不应有密钥、系统缓存或个人临时文件。若出现需要 Git 身份的错误，配置自己的身份后重试。

预期结果与成功检查

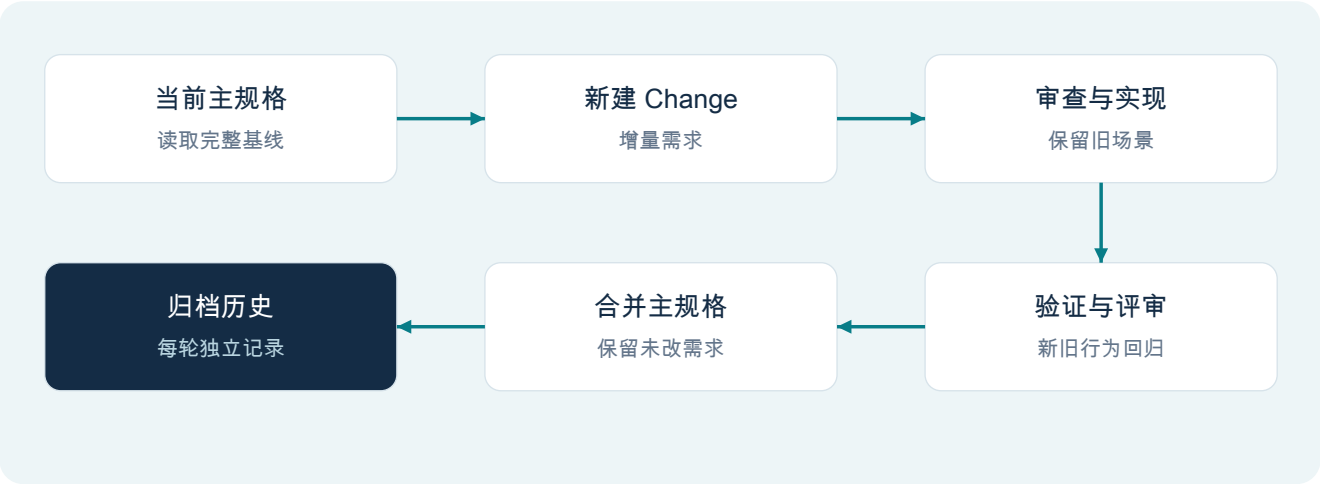
代码可以重新启动，主规格完整，历史目录不再被当作活跃任务。Git 提交保存了代码与知识，可通过历史追溯当时的提案和测试。

常见错误与解决

第二轮需求又去改第一轮 archive：改为新建 change，主规格是新 delta 的基线。想“回滚归档”却只移动目录：这不能回退代码或主规格，需按 Git 差异和业务状态完整处理。再次运行 init 不会替你增加新功能，下一轮从读取现有规格开始。

第二次开发：在已有系统上演进

本轮在第一轮 Todo 已归档且可运行的基础上，增加 low/medium/high 优先级和可选截止日期。继续使用 todo-items 能力，不建立重复的 todo-items-v2 规格。



操作目的、输入位置与命令

终端先确认基线，再建自己的工作分支：

```
git switch -c feat/todo-priority-due
openspec list --specs
openspec show todo-items --type spec
npm test
```

预期结果与成功检查

能读取主规格中全部第一轮需求，旧测试仍通过。此时只创建了 Git 分支，尚未创建 OpenSpec change。

常见错误与解决

找不到 todo-items：说明第一轮未同步或使用了另一能力名，先修正基线。已有未提交改动：先审查保存后再切分支。主规格与代码不同步：不要在不明确的基础上继续生成新 delta。

> 新变更名 add-todo-priority-due-date 表示一次工作；能力 todo-items 表示持续维护的行为。旧 archive 保持历史真实性，后续新增内容写入新 change。

先定义默认值与兼容行为

操作目的与输入位置

在 AI 聊天以既有规格为基线分析，再提出变更。不要把“加入两个字段”误解为推倒重写整个 Todo。

```
/opsx:explore
读取当前 todo-items 主规格和实现，评估新增优先级与截止日期。
优先级为 low/medium/high，默认 medium；日期可空，使用本地日历
日期 YYYY-MM-DD，允许过去日期，不设时间和通知。
旧任务 ID、标题、完成状态必须保留，补 medium 和空日期。
用户可新增或修改这两项；不引入排序、提醒和账号功能。
```

确认规则后在同一聊天继续：

```
/opsx:propose add-todo-priority-due-date
按已确认边界提出变更，复用 todo-items 能力路径。
先读完整主规格。仅对新增行为用 ADDED；已有行为变更用
MODIFIED 并保留该需求全部旧场景。保留其他需求不动。
包含旧数据兼容、日期校验、清空日期和回归测试。
```

预期文件与成功检查

新目录 changes/add-todo-priority-due-date/ 有 proposal、specs/todo-items/spec.md、design、tasks。proposal 的 Modified Capabilities 指向 todo-items，而不是随意新建近义能力。

```
openspec status --change add-todo-priority-due-date
openspec validate add-todo-priority-due-date --strict
```

常见错误与解决

助手给旧任务重分配 ID：要求保留 ID。用 UTC 时刻代替纯日期：修订为不带时区的日历日期。直接覆盖主 spec：将尚未确认的需求先放回 change delta，经过审查后再合并。

修改一条旧需求，增加两条新需求

本轮 MODIFIED 的“保存任务”复制第一轮完整需求，保留“刷新恢复”和“存储失败”两个旧场景，再补迁移场景。ADDED 写新增字段。其余“任务操作”“标题校验”既有需求不出现在 delta，但归档后仍应保留。

```
## MODIFIED Requirements

### Requirement: 保存任务
系统 SHALL 在存储可用时保留任务及新增字段，兼容旧记录，并在存储写入失败时继续支持本会话操作。
#### Scenario: 刷新恢复
- **WHEN** 用户添加或修改任务后刷新同一浏览器源的页面
- **THEN** 成功保存的任务 ID、标题与完成状态保持
#### Scenario: 存储失败
- **WHEN** 存储读写不可用或内容损坏
- **THEN** 页面不崩溃，无法读取时使用空列表，无法写入时提示刷新可能丢失修改
#### Scenario: 旧数据兼容
- **GIVEN** 旧记录只有 ID、标题和完成状态
- **WHEN** 新版读取并随后成功保存，再次读取
- **THEN** 原字段不变，优先级补 medium、截止日期补空值，重复读取不再改变数据

## ADDED Requirements

### Requirement: 优先级
系统 SHALL 支持 low、medium、high 三种优先级，默认 medium，并保留用户修改。
#### Scenario: 选择与修改
- **WHEN** 用户新增高优先级任务，再改为低优先级并刷新
- **THEN** 任务保留 low，未指定优先级的任务为 medium
#### Scenario: 非法优先级
- **WHEN** 输入不在枚举中的优先级
- **THEN** 拒绝该次修改，原任务数据不变

### Requirement: 截止日期
系统 SHALL 支持可选的有效 YYYY-MM-DD 日历日期，允许过去日期和清空，并在保存后保留。
#### Scenario: 设置与清空
- **WHEN** 用户设置有效截止日期后刷新，再清空并刷新
- **THEN** 第一次保留所设日期，第二次保持为空
#### Scenario: 非法日期
- **WHEN** 输入格式不符或不存在的日期，例如 2027-02-29
- **THEN** 拒绝该次修改，原任务数据不变
```

确认新需求不会覆盖历史承诺

操作目的与输入位置

在 AI 聊天中审查增量的一致性。编辑器同时打开当前主规格和新 delta，逐条核对名称与场景。

```
/opsx:update add-todo-priority-due-date
对比主规格和本次 delta：MODIFIED 必须包含保存任务的旧场景。
核查新字段默认为 medium 和空日期，迁移后保留 ID/标题/完成状态。
保持同一个存储键，不增加排序、提醒或账号功能。
日期用严格 YYYY-MM-DD 校验；清空日期允许；校验失败不提交修改。
给出变更审查结果及需要确认的修订，然后同步设计和任务。
```

预期结果：本轮设计要点

模型 createTodo 接受字段，updateTodo 修改字段；normalizeTodos 在读取时为旧记录补默认值。迁移先在内存中完成，下次成功保存时落盘，不在首次加载时破坏原数据。网页 form 增加两个控件，列表支持原地修改。使用 date 字段表示日历日期，不做跨时区时间运算。

可复制任务参考

```
## 1. 模型与兼容
- [ ] 1.1 校验优先级和日期，测试合法值、非法值与清空
- [ ] 1.2 兼容旧记录，验证 ID/标题/完成状态不变且迁移幂等
## 2. 界面与回归
- [ ] 2.1 新增与编辑字段，浏览器验证保存与刷新
- [ ] 2.2 保留全部第一轮测试并补新测试，验证全部通过
```

成功检查与常见错误

确认 tasks 包含数据兼容与旧功能回归。若新任务把旧测试删掉，要求恢复；若 MODIFIED 只有一句“增加日期”，回主规格复制整个需求块再改；若写“截止日期必须未来”，这属于未确认规则，应删掉或重新讨论。

用第一轮数据验证第二轮实现

操作目的与输入位置

在 AI 聊天让助手实施新 change。配套完整代码的 v2 与 v1 共用 index.html、app.mjs、style.css、server.mjs；第二轮替换 model.mjs，并扩展测试。

```
/opsx:apply add-todo-priority-due-date
按本轮计划实现，保留第一轮业务行为和已有测试。
请特别验证旧记录读取和保存后的字段兼容，不重建全部数据。
```

先在第一轮浏览器页面创建并勾选“保留这项旧任务”，然后停止 v1 服务器。切换到实现后的同一项目（或配套 v2 文件夹）再输入终端：

```
npm test
npm start
```

仍访问 http://127.0.0.1:5173，使用同一浏览器配置，才能读取原有数据。

预期文件与成功检查

旧任务出现，完成状态保留，优先级为“中”，日期为空。新增任务可选高优先级和有效日期，列表可修改和清空日期，刷新后结果保留。v2 参考项目包含 11 项模型测试，旧 6 项保留。

常见错误与解决

数据为空：检查端口、主机名和浏览器配置是否改变，别急着清空存储。模型正确但控件未出现：确认 extended=true 的新版 model 已加载，刷新页面并检查网络。日期非法输入：应用应拒绝并提示，不能将无效日期写入存储。

同时验证新需求和旧需求

操作目的与输入位置

终端做结构与回归检查，AI 聊天做规格对照并归档。这里的两个助手操作按顺序执行，先读 verify 报告并解决问题，再发 archive。

```
npm test
openspec validate add-todo-priority-due-date --strict
```

```
/opsx:verify add-todo-priority-due-date
请按主规格与本轮 delta 核查全部需求。
重点检查：旧数据 ID/标题/完成态保留，字段默认值，
闰年和非法日期，清空日期，刷新持久化，以及旧增删完成操作。
列明实际证据和未验证项，先不要归档。
```

确认通过后：

```
/opsx:archive add-todo-priority-due-date
同步本轮 delta 后归档。保留主规格中未被本轮修改的需求，
汇报 MODIFIED 与 ADDED 的合并结果。
```

预期结果与成功检查：终端

```
openspec show todo-items --type spec
openspec validate --all --strict --no-interactive
openspec validate --archived --no-interactive
```

主规格最终有五条需求：任务操作、标题校验、保存任务、优先级、截止日期。“保存任务”有原两个场景与新迁移场景；历史中保留两轮独立变更目录。

常见错误与解决

主规格只有新字段，旧功能消失：错误地整文件覆盖了 delta，应依据 Git 与基线恢复完整主规格，再正确合并。旧迁移仅跑一次正常、重读又变化：补幂等测试，读取新版记录不应再丢字段。

规格演进的结果应能解释清楚

操作目的与输入位置

编辑器比较主规格和两轮历史；终端查看差异并提交。当前主规格描述完整产品，旧 delta 只保留当时新增/修改内容。

```
openspec/
  specs/todo-items/spec.md
    任务操作 + 标题校验 + 保存任务（含迁移）
    + 优先级 + 截止日期
  changes/archive/
    DATE-add-todo-list/              第一轮历史
    DATE-add-todo-priority-due-date/   第二轮历史
```

可复制命令：终端

```
git diff -- openspec/specs/todo-items/spec.md
git status --short
git add .
git commit -m "feat: add todo priority and due date"
```

团队项目应先走后文 PR 流程。配套 v2 保留两份完整历史，读者可以直接比较第一轮主规格如何演进。

成功检查

旧需求标题仍存在，原场景未被缩写删除；新增字段、默认值和兼容行为均有场景；两轮 tasks 仅在完成后勾选，归档校验通过；npm test 与页面操作正常。

常见错误与解决

把 MODIFIED 当附加说明：归档是替换目标需求块，必须包含完整新版本。看到 archive 就以为改动已合并：它只操作文件，仍需 Git 提交/PR。优先级排序、到期通知等新想法应另建变更，不在本轮最后临时塞入。

登录、支付、搜索并行开发

下面模拟一个已有应用仓库 team-app，远端分支为 main，已具备测试和 OpenSpec。三人各有独立 clone；这里展示的是协作流程，不在 Todo 项目中真正接入登录或真实支付服务。支付只讨论规格与模拟适配器，不涉及真实扣款。



开发者	Git 分支	OpenSpec Change	主能力
Alice	feat/login	add-user-login	user-auth
Bob	feat/payment	add-checkout-payment	checkout-payment
Chen	feat/search	add-product-search	product-search

共同约定

每个 change 有单一负责人；代码和 delta 一起进 PR；PR 内先审计划再审代码；合并后以小型归档 PR 更新共享主规格。共享的 platform-errors 能力由评审者协调，不能让三人独立覆盖同一需求。

输入位置与成功检查

每人的终端先 git switch main、git pull --ff-only、openspec update，然后在自己的聊天会话中运行相应 propose。看到分支名不同、change 目录不同，且彼此工作区互不覆盖即完成准备。

常见错误

三人共用同一未隔离工作目录：文件会互相影响，应各自 clone 或用 Git worktree。同一 change 被多人随意勾选：拆分责任或由负责人整合，不能把复选框当任务分配系统。[R15]

分支与 Change 一一对应

终端：三人分别执行自己的命令

```
# Alice 的独立工作区
git switch main
git pull --ff-only
git switch -c feat/login
# Bob、Chen 在各自工作区把分支换成表中的名字
```

AI 聊天：分别提出和实施计划

```
/opsx:propose add-user-login
已有应用增加登录：成功、错误凭据、会话过期与退出。
复用 user-auth，明确统一错误格式，先生成计划等待评审。
```

Bob 改为 add-checkout-payment：定义模拟支付成功、失败、重复请求幂等与待确认结果，不包含真实网关密钥。Chen 改为 add-product-search：定义关键字、分页、空结果和非法查询，明确鉴权与错误格式边界。

审查计划后，每人分别调用 /opsx:apply <自己的变更名>；终端执行该项目的 npm test 和 openspec validate <自己的变更名> --strict，再让 verify 对照规格与证据。

预期结果与成功检查

每个分支只包含本功能相关代码、测试与自己的变更目录。登录、支付、搜索的能力边界清楚，接口使用共同约定；未实现别人的需求时，不勾选别人的任务。

常见错误与解决

Bob 等登录主分支先完成才开始写：可先约定鉴权接口并用模拟适配器开发，在集成任务中验证真实依赖。三个 change 都叫 add-feature：改为明确名称。助手开始自动提交/推送：Git 操作由开发者按团队权限明确执行。

Bob 的支付分支与 PR

终端：在 Bob 自己的 clone 中准备

```
git switch main
git pull --ff-only
git switch -c feat/payment
openspec update
```

AI 聊天：规划、审核与实现

```
/opsx:propose add-checkout-payment
在既有应用上增加模拟支付，能力为 checkout-payment。
覆盖成功、明确失败、结果待确认和重复请求幂等。
引用已有 user-auth 与 platform-errors 约定。
不接真实网关，不处理真实资金；输出计划后等待审查。
```

审核 proposal、delta、design、tasks。若登录接口尚未合并，设计应明确模拟适配与后续集成验收。批准计划后在聊天框输入：

```
/opsx:apply add-checkout-payment
```

终端运行下面检查，再在 AI 聊天调用 /opsx:verify add-checkout-payment，解决发现的问题：

```
npm test
openspec validate add-checkout-payment --strict --no-interactive
git diff
git add .
git commit -m "feat: add checkout payment"
git push -u origin feat/payment
gh pr create --base main --head feat/payment --web
```

预期结果、成功检查与排错

网页 PR 选择 feat/payment → main，填写实际测试结果与模拟支付范围。若 user-auth 已变化，先更新基线并重新集成测试。功能 PR 合并后，按“合并后的规格归档”另建 chore/archive-payment，调用 /opsx:archive add-checkout-payment，校验、提交并发归档 PR。

若出现平台错误规格冲突，先按后文案例解决；不要将支付 delta 直接覆盖到整份主规格。

Chen 的搜索分支与 PR

终端：在 Chen 自己的 clone 中准备

```
git switch main
git pull --ff-only
git switch -c feat/search
openspec update
```

AI 聊天：规划、审核与实现

```
/opsx:propose add-product-search
在既有应用增加搜索，能力为 product-search。
明确关键字处理、分页、空结果、非法参数的可验收行为。
复用已有鉴权与 platform-errors 错误约定，避免新建近义规格。
先读相关主规格，再生成计划，不实现支付或登录页面。
```

审查范围与边界后在 AI 聊天输入：

```
/opsx:apply add-product-search
```

先运行终端检查，再调用 /opsx:verify add-product-search。报告中的成功结论应有代码和测试证据：

```
npm test
openspec validate add-product-search --strict --no-interactive
git diff
git add .
git commit -m "feat: add product search"
git push -u origin feat/search
gh pr create --base main --head feat/search --web
```

预期结果、成功检查与排错

PR 目标为 main，包含搜索行为 delta 与实现测试，不能混入支付工作文件。支付或登录先合并时，Chen 更新 origin/main 后重新核对共享约定，不能只检查自己的代码能编译。

搜索 PR 合并后，在最新 main 上另建 chore/archive-search，调用 /opsx:archive add-product-search，严格校验并提交归档 PR。最后确认三个能力存在、三个 change 均归档、历史任务校验和业务测试通过。

PR 里同时审“要做什么”和“实际做了什么”

操作目的与输入位置

各自终端提交并推送自己的分支；随后在 GitHub 网页创建 PR，或已安装 GitHub CLI 时运行 gh pr create。这些是 Git/GitHub 命令，不属于 OpenSpec。

```
git status --short
git diff
git add .
git commit -m "feat: add user login"
git push -u origin feat/login
gh pr create --base main --head feat/login --title "Add user login" --body-file pr-body.md
```

以上 Alice 示例。Bob、Chen 替换分支和标题。执行 gh 命令前，在 pr-body.md 中写入下面内容；它是 PR 描述文件，不是 OpenSpec 专用格式。

```
## 变更
实现登录功能；计划见 openspec/changes/add-user-login/。
## 评审重点
先看 proposal 与 delta，再检查设计、代码和测试。
重点核对会话过期、凭据失败与统一错误格式。
## 验证
填写实际执行的测试命令、结果与未验证事项。
## 后续
合并后由负责人发归档 PR，同步主规格。
```

预期结果与成功检查

main 同时有三个待评审 PR。每个 PR 显示源分支正确、变更范围合理、测试通过，评审者能对应需求场景找到代码和测试证据。

常见错误与解决

gh 不存在/未认证：用 GitHub 网页建 PR。PR 含另一个开发者文件：检查工作区和分支来源，拆开误提交。计划 PR 已通过但代码改变行为：更新计划并重新评审，不把旧批准当永久认可。

先集成实现，再更新共享主规格

操作目的与输入位置

本例选“主功能 PR 合并后，另发归档 PR”的团队约定。maintainer 先合并 Alice 的 PR；Alice 更新 main 后，在新分支归档，避免直接改受保护 main。

```
git switch main
git pull --ff-only
git switch -c chore/archive-login
```

AI 聊天：

```
/opsx:archive add-user-login
实现已随 PR 合并。请核对当前主分支代码与计划，
同步 delta 并归档，报告统一错误格式是否有冲突。
```

终端检查并提交：

```
openspec validate --all --strict --no-interactive
openspec validate --archived --no-interactive
npm test
git add openspec
git commit -m "docs: archive user login change"
git push -u origin chore/archive-login
```

网页为此分支创建归档 PR。该 PR 合并后，Bob 和 Chen 先更新自己的工作分支，再处理剩余功能 PR 和归档 PR。

预期结果与成功检查

主规格只吸收已经集成的代码行为，历史可追溯。每次归档 PR 基于最新 main，原活跃 change 只归档一次。

常见错误与解决

旧分支又归档同一 change：先 pull 最新 main，确认活跃目录是否还在。多个归档同时改同一能力：串行合并并重新核对 delta。团队也可选择在功能 PR 内归档，但应统一约定，不混用时机。

同一旧需求被两个完整块替换

共享能力 platform-errors 原有需求如下；Alice 和 Bob 都在各自 delta 中使用 MODIFIED，语法合法却可能互相遗漏。即使 Git 能自动合并，也必须识别语义冲突。

```
### Requirement: 错误响应
系统 SHALL 在错误响应中返回 code 与 message。
#### Scenario: 非法请求
- **WHEN** 请求参数无效
- **THEN** 返回包含 code 与 message 的错误响应
```

Alice 完整替换此块，新增“未登录返回 401”。Bob 基于旧版本完整替换此块，新增“支付失败返回业务错误码”。如果 Bob 最后同步旧块，可能把 Alice 的 401 场景覆盖掉。

操作目的、位置与可复制提示

在 Bob 的工作分支终端先更新基线，冲突时暂停检查：

```
git fetch origin
git merge origin/main
```

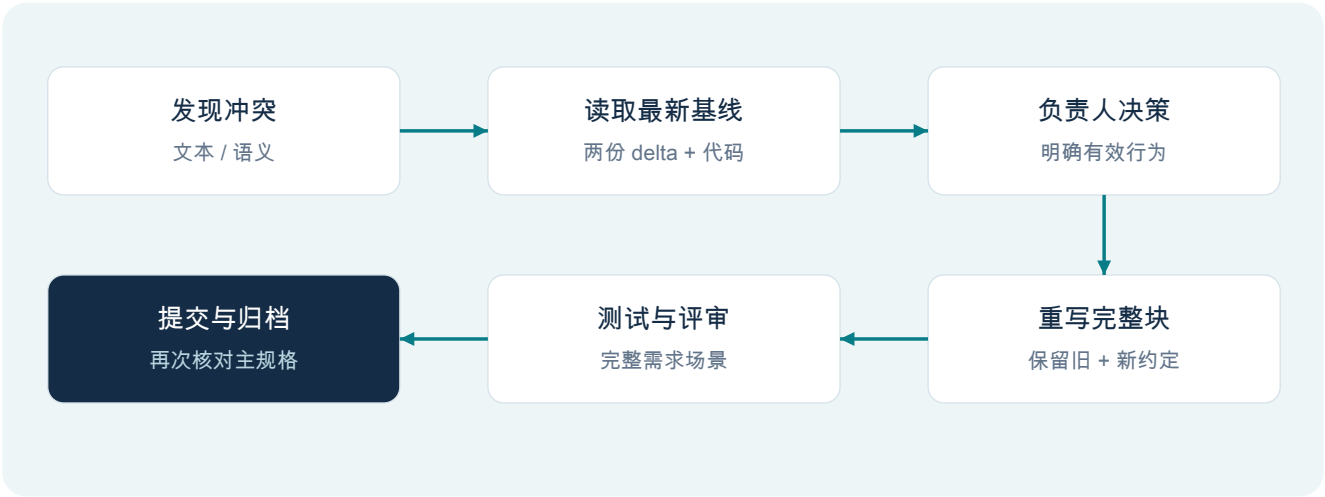
然后在 AI 聊天输入：

```
读取最新 platform-errors 主规格、登录历史 delta 和支付 delta。
比较同名“错误响应”的完整需求块，找出丢失的登录场景。
提出保留原参数校验、登录 401、支付错误码三者的完整版本。
不要采用只保留 ours/theirs 的整文件覆盖；先给差异与决策依据。
```

成功检查与常见错误

区分文本冲突和语义冲突；核对业务接口是否支持两种错误场景。若登录与支付约定彼此矛盾，先由负责人决定规则再改代码和规格；“最后写入者优先”不是业务决策。[R10][R15]

合并意图，保留全部仍有效的场景



合并后的完整需求块示例

```
### Requirement: 错误响应
系统 SHALL 在错误响应中返回 code 与 message。
#### Scenario: 非法请求
- **WHEN** 请求参数无效
- **THEN** 返回包含 code 与 message 的错误响应
#### Scenario: 未登录
- **WHEN** 未登录用户请求受保护资源
- **THEN** 返回 HTTP 401 与 UNAUTHENTICATED 错误码
#### Scenario: 支付失败
- **WHEN** 支付适配器明确返回支付失败
- **THEN** 返回 PAYMENT_FAILED 错误码且不报告支付成功
```

把这个完整块放进 Bob 的 MODIFIED delta，在确认实现与新基线一致后处理 Git 冲突标记，并重新测试。没有文本冲突时仍执行相同的语义核对。

```
git add openspec
# 若当前处于 merge 冲突解决阶段，完成合并提交
git commit
openspec validate add-checkout-payment --strict
npm test
```

AI verify → 支付功能 PR 合并 → 新归档 PR，同步时再次以最新主规格检查。搜索分支同样更新基线并验证；三个功能结束后检查主能力、活跃变更列表和归档任务。

> bulk-archive 可以辅助分析多个变更，但不代表自动批准业务冲突。结构校验也不能证明登录 401 场景未丢失，必须审查合并后的完整需求块。

为现有网页增加深色模式

案例假设：已有可以启动的网页项目，本书不限定 React、Vue 或原生 HTML。先在工作分支或可回退的目录上试做。下面的文档片段是你应当审查的目标产物，并非承诺助手每次输出完全相同。[R4][R10]

第一步：调查现状

```
/opsx:explore
检查项目现有主题、样式入口和测试方式。
我要增加浅色/深色切换。请先比较最小改动方案，
说明刷新、系统主题变化、存储异常会如何处理。
```

确定本次边界：初次打开随系统；手动选择优先并在可用时持久化；存储失败时仍可切换。暂不增加第三个“跟随系统”按钮，不做多端账户同步。

第二步：提出变更

```
/opsx:propose add-dark-mode
沿用现有技术栈。按刚才确认的边界生成计划，
保留英文结构标题，中文编写正文，并列出测试方式。
```

proposal.md 的可用示例

```
## Why
夜间使用时界面过亮，需要可持续生效的深色模式。

## What Changes
- 首次加载遵循系统主题。
- 提供主题切换；保存手动选择，存储失败时降级。
- 覆盖现有主要页面和键盘操作。

## Capabilities
### New Capabilities
- theme: 主题选择、偏好保留和系统主题响应
### Modified Capabilities
无。

## Impact
影响全局颜色变量、页面启动逻辑、主题按钮与测试。
不改变后端接口，不增加账户级主题同步。
```

先把边界写成场景

保存到 openspec/changes/add-dark-mode/specs/theme/spec.md。新能力 Purpose 描述其长期职责；Delta 使用 ADDED。已有 theme 能力时应先读取主规格，再判断哪些内容属于 MODIFIED。

```
## Purpose
定义网页的主题选择与偏好保留行为，使用户在不同
系统主题和浏览器存储条件下获得一致、可解释的显示
结果，并为后续主题相关功能提供稳定的行为约定。

## ADDED Requirements

### Requirement: 初始主题
没有有效手动偏好时，页面 SHALL 使用系统主题。
#### Scenario: 首次访问且系统为深色
- **GIVEN** 没有已保存的有效偏好
- **WHEN** 系统偏好为深色且页面加载
- **THEN** 页面使用深色
#### Scenario: 系统主题变化
- **GIVEN** 本次会话没有手动选择且无有效已存偏好
- **WHEN** 系统主题从浅色变为深色
- **THEN** 页面切换到深色

### Requirement: 手动主题选择
用户 SHALL 能切换主题，且手动选择优先于系统主题。
#### Scenario: 手动浅色优先
- **GIVEN** 用户已选择浅色
- **WHEN** 系统切换为深色
- **THEN** 页面仍为浅色

### Requirement: 主题偏好持久化
系统 SHALL 尝试保存手动选择，存储失败不得阻止切换。
#### Scenario: 刷新保留选择
- **GIVEN** 深色选择已成功保存
- **WHEN** 用户刷新页面
- **THEN** 页面仍为深色
#### Scenario: 浏览器禁止存储
- **GIVEN** 读写存储会抛出异常
- **WHEN** 用户选择深色
- **THEN** 页面立即变深且功能继续可用
- **AND** 本会话中不被系统主题变化覆盖
```

> 按本例补充：非法存储值视为无有效偏好；按钮必须可用键盘触发且有可理解名称。这些应写进正式规格，避免仅存在于聊天中。

让规格落到清晰的技术决策

本例采用 CSS 自定义属性 + 根元素 data-theme + localStorage。理由：小型网页无需新依赖，可复用既有样式入口。系统偏好由 matchMedia 提供；读写存储均捕获异常。复杂组件库项目优先使用其已有主题方案。

```
## 1. 主题决策
- [ ] 1.1 实现偏好解析与主题决策，测试有效值、非法值和系统回退
- [ ] 1.2 接入启动和系统主题监听，验证无手动选择时跟随系统

## 2. 交互与持久化
- [ ] 2.1 按钮支持键盘与可访问名称，验证切换后颜色和名称
- [ ] 2.2 存储异常时保留会话选择，测试刷新和异常路径
- [ ] 2.3 更新主要页面色值，逐页检查文字、边框和焦点可见性

## Workflow follow-up
- 评审、合并后按团队约定同步规格并归档。
```

可直接运行的决策函数：theme.mjs

```
export function resolveTheme(saved, systemDark) {
    if (saved === "light" || saved === "dark") {
        return saved;
    }
    return systemDark ? "dark" : "light";
}

export function saveTheme(storage, theme) {
    try {
        storage.setItem("theme", theme);
        return true;
    } catch {
        return false;
    }
}
```

这只实现决策与写入降级。完整网页还需要安全读取、会话内手动偏好、系统事件监听、data-theme 和按钮集成；将它们交给 apply 按任务完成。不能用这两个函数的测试代替页面验收。

把“完成”落实到可检查的结果

在新会话调用 /opsx:apply add-dark-mode；要求助手先读取变更目录和项目约束。下面是 theme.test.mjs，可与上一页函数放在同一目录，以 node --test 运行（Node ≥ 20.19.0）。

```
import test from "node:test";
import assert from "node:assert/strict";
import { resolveTheme, saveTheme } from "./theme.mjs";

test("手动偏好优先", () => {
  assert.equal(resolveTheme("light", true), "light");
  assert.equal(resolveTheme("dark", false), "dark");
});
test("非法偏好回退到系统", () => {
  assert.equal(resolveTheme("unknown", true), "dark");
  assert.equal(resolveTheme(null, false), "light");
});
test("禁止存储时返回失败状态", () => {
  const storage = {
    setItem() {
      throw new Error("blocked");
    }
  };
  assert.equal(saveTheme(storage, "dark"), false);
});
```

页面验收	预期结果
清除偏好后首次打开	随系统；非法存储值也回退
手动选浅色后改变系统主题	保持浅色；刷新后仍保持（存储可用）
禁止存储后点击按钮	页面切换成功；本会话不被系统覆盖
键盘 Tab 与 Enter/Space	可聚焦、可触发、名称清楚
逐页检查深色主题	文字、链接、边框、焦点均可辨认

终端运行 openspec validate add-dark-mode --strict，按项目方式执行测试与构建。安装了 verify 再运行 /opsx:verify add-dark-mode。审查代码与计划一致后，用 /opsx:archive add-dark-mode 确认同步；然后检查 openspec/specs/theme/spec.md 与归档目录。

按行为变化选择文档规模

Bug 修复：回到既有承诺

假设规格已要求“存储异常时仍可切换”，但实现直接抛错。先复现，再修实现与测试；若规格已经正确，不必把同一句话再次 ADDED。若原规格确实漏掉异常约束，则建立变更补充需求和场景。[R10][R11]

```
/opsx:propose fix-theme-storage-failure
主题写入失败会阻止切换。先读取现有 theme 规格，
判断是实现违约还是规格缺口，避免重复新增需求。
```

纯重构：不改变对外行为

例如拆分主题模块但全部行为保持不变，创建变更后在 .openspec.yaml 中明确写入：

```
schema: spec-driven
skip_specs: true
```

保留提案与可验证的任务。运行 status 应看到 specs 被跳过；结构校验无需你杜撰一份“代码更清晰”的行为规格。只在确信外部行为没有变化时使用此标记。

变更已经开始，需求又改了

小范围修订：update 修订现有文档 → 审核新增/失效任务 → apply 继续。实现方案改了但行为不变，主要调整 design 与 tasks；行为变了，先调整 specs。

意图已经改变：新建变更，说明与旧变更的关系。已经归档的功能再次修改时，在当前主规格基础上新建 delta，保留历史的真实性。

取消、回滚与归档的区别

未实施变更可以按团队约定删除或记录取消原因。已部署功能回滚时，代码回退还要让当前规格恢复对应行为；宜建立新的回滚变更。archive 只是记录完成的计划，不能撤销代码或部署。

> 团队建议：不要只为“进度好看”勾选没做的任务；调整范围时明确记录取消或拆分原因，并重新核对验收。

终端命令速查：设置

以下覆盖 v1.15.0 的公开 CLI 命令组与主要可用子命令。参数以本机 openspec <命令> --help 为最终依据；尖括号是占位符，执行时替换。[R12]

命令	中文用途与主要参数
openspec --help	根帮助；help <command> 查看某命令；-h 是简写
openspec --version	输出版本，简写 -V；--no-color 关闭彩色输出
openspec version	显示安装版本/来源；--check 联网检查更新；--json 结构化输出
openspec install [version]	安装或更新独立版本；不给版本时选择最新可用发布
openspec init [path]	初始化指定项目，默认当前目录
openspec update [path]	根据本机 profile/delivery 刷新项目助手文件

init 的主要选项

--tools <id,id>：逗号分隔工具 ID，可用 all 或 none。常见 ID 为 codex、claude、cursor、gemini、github-copilot、antigravity、opencode、agents。

--language <language>：为新配置添加语言指令。--profile core|custom：本次初始化的工作流集合。

--no-animation：禁用欢迎动画。--force：不询问便清理旧版文件，应先审查现有内容。

--copilot-cloud / --no-copilot-cloud：明确启用/跳过 Copilot 云端文件设置。

update 的主要选项

--dry-run 只预览新增、修改和删除，--dry-run --json 输出机器可读计划。--force 强制重写当前生成文件。单独 --json 不受支持，应与 --dry-run 合用。

```
openspec update --dry-run --json
openspec update --dry-run --force
openspec update
```

> 升级前先确认安装来源、审查生成文件的本地修改。update 的版本检查可能提示升级 CLI；离线或不希望检查时，可设置 OPENSPEC_NO_UPDATE_CHECK=1。

终端命令速查：日常维护

命令	中文用途与主要参数
openspec list	列活跃变更；--specs 列能力；--archived 只列历史；--all 含活跃与历史；--sort recent/name；--json
openspec view	显示终端概览，便于查看规格与变更
openspec show [item]	查看变更或能力；同名时 --type change/spec；--json；--no-interactive
openspec validate [item]	检查结构与 delta；--all / --changes / --specs；--strict；--json
openspec archive [name]	校验、更新主规格、移动归档；-y/--yes 跳过确认

show 的过滤能力

变更用 --deltas-only（JSON 模式）只看增量，--diff 显示逐需求差异；规格用 --requirements 或 --no-scenarios（JSON）省去场景，-r/--requirement <序号> 查看某条需求。概要筛选后，做语义判断前仍应读完整相关规格。

```
openspec list --specs
openspec show theme --type spec
openspec show theme --type spec --json --no-scenarios
openspec validate add-dark-mode --strict --no-interactive
openspec validate --all --strict --json
openspec validate --archived
```

validate 还支持 --type、--report full/findings（批量报告详略）和 --concurrency <数量>，默认并发 6。--archived 是独立检查范围，检查历史变更中的任务是否全部完成，不重新应用或校验已同步 delta；--all 不等于包含这项检查。

archive 的边界参数

--yes 用于已确认目标和效果的非交互调用，必须明确变更名；--json 返回结构化归档结果。--skip-specs 仅跳过本次规格更新；长期没有行为 delta 的变更应使用元数据 skip_specs。--no-validate 跳过校验且需额外确认，不宜作为日常消除错误的方法。

归档通常生成 changes/archive/YYYY-MM-DD-<name>/。目标重名或 delta 不合法时应先排错。CLI 支持失败恢复保护，仍应通过版本管理检查实际结果。[R12]

这些命令多由 AI 助手使用

命令	中文用途与主要参数
openspec new change <name>	建立变更骨架；--schema；--description；--goal；--json；--store
openspec status	查产物状态；--change <id> 指定变更；--all 看全部；--json；--schema
openspec instructions [artifact]	输出模板与执行输入；--change；--json；--schema
openspec templates	查看解析后的模板路径；--schema；--json
openspec schemas	列可用 schema、来源、描述与流程；--json

手动观察一次计划生成过程

```
openspec new change add-dark-mode
openspec status --change add-dark-mode
openspec instructions proposal --change add-dark-mode
openspec instructions specs --change add-dark-mode --json
openspec instructions design --change add-dark-mode --json
openspec instructions tasks --change add-dark-mode --json
```

新建后 proposal 应就绪，下游受依赖约束。instructions 只是把“该写什么”返回给调用者；你或助手仍要按其内容编写产物。不要把输出 JSON 原样保存成 proposal.md。

apply 与 archive 也是 instructions 的输入界面

```
openspec instructions apply --change add-dark-mode --json
openspec instructions archive --change add-dark-mode --json
```

前者提供实施上下文及任务；后者只读取归档的当前 context 与 operationGuidance 等输入，不执行移动、不合并主规格。真正归档用 archive 命令或助手归档工作流。

变更名规则

使用小写字母、数字和单个连字符，如 add-dark-mode 或 100-add-auth。禁止空格、下划线、大写、连续连字符和首尾连字符。中文需求正文可以保留中文，目录名建议用可读英文。

> 自动化调用显式指定 --change、--type 与 --no-interactive（命令支持时），并检查退出码和 JSON 内容，避免脚本等着交互输入。[R12]

全局偏好与项目工作流分开管理

config 子命令	中文解释
path	显示全局 config.json 的真实位置
list [--json]	列出当前设置与 profile 概要
get <key>	读取一个键，方便脚本使用
set <key> <value>	写入值并进行类型转换；--string 强制字符串
unset <key>	删除键，恢复该键的默认行为
reset	重置配置；先用 --help 确认范围，--all --yes 重置全部
edit	用 $EDITOR 打开全局配置
profile [preset]	交互选择 delivery/工作流；preset core 恢复默认六项

全局文件：macOS/Linux 通常在 ~/.config/openspec/config.json，Windows 在 %APPDATA%\openspec\config.json。XDG_CONFIG_HOME 可影响位置，用 config path 最稳妥。当前 --scope 只支持 global，项目上下文应编辑 config.yaml。[R7][R12]

schema 子命令	中文解释与主要参数
init <name>	建项目本地 schema；--description；--artifacts；--default/--no-default；--force；--json
fork <source> [name]	复制已有 schema 到项目中；--force；--json
validate [name]	检查 schema、模板和依赖；--verbose；--json；省名检查全部
which [name]	查看 schema 解析来源；--all；--json

```
openspec config profile
openspec update
openspec schema fork spec-driven team-flow
openspec schema validate team-flow
openspec schema which team-flow
openspec new change add-search --schema team-flow
```

--force 可能覆盖文件，先检查目标目录。schema 为实验性接口，团队采用后应锁定 CLI 版本，将 schema 与模板一起纳入版本控制，升级时检验兼容性。

补全、反馈、兼容命令

命令	中文解释
openspec completion generate [shell]	向标准输出生成补全脚本
openspec completion install [shell]	安装当前或指定 shell 的补全
openspec completion uninstall [shell]	卸载补全，宜在卸载 CLI 前执行
openspec feedback <message>	提交问题反馈；可能调用浏览器/GitHub，不用于自动反馈项目隐私内容
openspec change show [name]	旧式查看变更入口，改用 show --type change
openspec change list	旧式列变更入口，改用 list
openspec change validate [name]	旧式变更检查，改用 validate --type change
openspec spec show [id]	旧式查看规格入口，改用 show --type spec
openspec spec list	旧式列规格入口，改用 list --specs
openspec spec validate [id]	旧式规格检查，改用 validate --type spec

补全支持 Bash、Zsh、Fish 与 PowerShell。安装/卸载的具体选项用该子命令 --help 查看。Nix 包已带标准位置的补全脚本，一般无需重复安装。[R3][R12]

常用环境变量

名称	中文解释
OPENSPEC_TELEMETRY=0	关闭匿名用量统计，也可关闭相关更新检查
DO_NOT_TRACK=1	关闭匿名统计的另一入口
OPENSPEC_NO_UPDATE_CHECK=1	跳过更新检查
OPENSPEC_CONCURRENCY	校验命令默认并发数量
OPENSPEC_INSTALL_DIR	独立安装脚本的目标目录
OPENSPEC_VERSION	安装脚本固定版本，如 v1.15.0

旧教程迁移提醒

旧版 /openspec:proposal、/openspec:apply、/openspec:archive 不是当前新项目的推荐入口。升级前检查旧集成，先运行 update --dry-run；不要假定旧命令会自动存在。变更文档通常仍可在 OPSX 中继续，迁移后核对本机帮助与实际生成文件。

一页核对 v1.15.0 的命令集合

以下覆盖根帮助与公开命令组的可见入口，不列内部补全实现。每行在前面加 openspec；参数、应用场景和注意事项见相邻 CLI 分节、配置篇及 Stores/Worksets 篇。所有入口可用 --help 查询本机帮助。[R12]

类别	命令与子命令	中文职责
版本与安装	version；install [version]；--version/-V	版本/来源、检查更新、独立安装
项目集成	init [path]；update [path]	初始化与刷新生成指令
浏览	list；view；show [item]	列出、概览、读取对象
校验与归档	validate [item]；archive [name]	结构检查、合并规格与归档
产物流程	new change <name>；status；instructions [artifact]；templates；schemas	骨架、状态、指导输入、模板、schema 清单
全局配置	config path/list/get/set/unset/reset/edit/profile	定位、读写、重置和选择工作流
自定义流程	schema which/validate/fork/init	来源解析、校验、复制与创建
规格仓库	store setup/register/unregister/remove/list/doctor	创建、注册、忘记、删除、列表与诊断
工作根与关系	doctor；context	关系诊断与组装工作上下文
个人工作集合	workset create/list/open/remove	创建、列出、重开和删除个人视图
反馈与补全	feedback <message>；completion generate/install/uninstall	反馈、生成/安装/移除 shell 补全
旧式入口	change show/list/validate；spec show/list/validate	兼容入口，优先使用动词命令
帮助与别名	help [command]；各组 help；store ls；workset ls	帮助；ls 是相应 list 的别名

不能混淆的入口

不存在 openspec propose、openspec apply 或 openspec verify 这样的官方终端命令。对应操作在 AI 聊天调用工作流；CLI 的 instructions apply 只给实施输入，不替你实现业务代码。

参数书写说明

命令表中的斜线列举同组子命令，例如 config get，并不是输入 config path/list/get。--type 的取值是 change 或 spec；--sort 的取值是 recent 或 name。不要把中文用途、方括号或尖括号原样输入终端。

让 AI 每次都读到必要的约束

配置放 openspec/config.yaml。schema 选择工作流；context 传达持续有效的事实；rules 为指定产物补充内容规则；operations 给 apply/archive 添加操作建议。[R13]

```
schema: spec-driven
context: |
  本项目是已有网页产品，保留现有技术栈。
  产物正文使用中文；保留 OpenSpec 英文结构标题。
  公共接口保持兼容，新增行为必须有可验收场景。
rules:
  proposal:
    - 写清本次范围与不做的内容
  specs:
    - 包含与需求相关的异常和边界场景
  design:
    - 解释关键技术选择与替代方案的取舍
  tasks:
    - 每个任务包含完成证据或验证方法
operations:
  apply:
    guidance:
      - 优先运行与当前修改相关的测试
  archive:
    guidance:
      - 汇报规格同步结果与未解决事项
```

配置	生效范围	注意
context	产物生成、apply、archive	只放必要事实，最长 50KB；过大忽略并警告
rules.<artifact>	对应产物的生成	与内置指导叠加；verify 不读取这类规则
operations.apply/archive.guidance	对应操作	是建议，不能替代强制检查
schema	默认工作流	不会批量改写已有变更的元数据

规则多了不一定更可靠

详细的永久文档保存在仓库，用简短上下文指向它们。不要把整个 README、代码库或所有历史决策塞入 context。修改配置后，下次 instructions 会重新读取；配置中的自然语言约束没有自动执行的强制能力，关键要求需工具校验。

先改项目规则，再考虑改流程

默认产物已经足够时用 config.yaml。只有确实需要新的产物、依赖或模板时才 fork schema。下面演示增加“实施前评审记录”，它是流程示例，不是自动审批系统。[R14]

```
openspec schema fork spec-driven team-flow
```

在复制得到的 schema.yaml 中，保留原产物，并在 tasks 之前加入 review，修改 tasks 的 requires（不要保留两个同名 tasks）：

```
  - id: review
    generates: review.md
    description: 实施前风险评审记录
    template: review.md
    instruction: |
      记录范围、测试和兼容性检查结果及待确认事项。
    requires:
      - specs
      - design

  # 在原 tasks 定义中仅替换 requires
  # requires:
  #   - specs
  #   - design
  #   - review
```

创建 templates/review.md，给出“结论、风险、待处理事项、评审人”等标题。上面的片段不是完整 schema；直接复制粘贴前要保持与原 artifacts 列表的缩进一致。

```
openspec schema validate team-flow
openspec new change add-theme --schema team-flow
openspec status --change add-theme --json
```

两套优先级

选 schema 名称：显式 --schema → 变更 .openspec.yaml → 项目 config.yaml → spec-driven。

解析同名 schema 的文件：项目 openspec/schemas/ → 用户数据目录 schemas/ → 内置。用 schema which 检查覆盖来源。

> review.md 存在不表示负责人真的批准。需要人工签署或测试通过才允许实现/合并时，用评审与 CI 实现；仅增加依赖文件无法形成硬门禁。

让规格和代码一起接受评审

OpenSpec 的日常工作流读写项目文件，不替你创建业务分支、提交、推送或合并。store setup 默认可初始化新 store 的 Git 仓库，这是显式设置行为，应与日常工作区分。[R15]



推荐约定：一个变更，一个负责人

变更目录跟随业务分支，PR 同时包含计划、代码、测试与必要文档。评审按 proposal → specs → design/tasks → 代码证据顺序阅读。分工不同也保持一位负责人维护同一变更的整体一致性。

角色	重点责任
需求/产品	行为、范围、异常与验收是否符合真实需要
开发负责人	方案、任务、实现和迁移是否可执行
评审者	需求与实现对应，风险与测试证据充分
CI	固定版本、结构校验、测试与构建

归档时机选择一个并统一

团队可在合并后用跟进 PR 同步归档，也可在原 PR 内完成。前者使主规格只吸收已合并工作，但会多一次提交；后者流程简短，评审需同时看规格合并结果。归档不是“已上线”的自动证明。

两个变更修改同一需求

更新分支、读取最新主规格，比较两个 delta 的意图与已实现代码，形成完整的新需求块，再校验与测试。不要机械保留最后写入的一份。bulk-archive 可以帮助检查冲突，但业务取舍仍需负责人确认。

当一份计划需要服务多个代码仓库

Store 是单独存放 OpenSpec 规格和变更的仓库，通过本机注册后可从其他目录选用。先从单仓库起步；有跨服务计划、平台契约共享时再引入。[R16]




```
# 建立本地 store，不执行 Git 初始化
openspec store setup team-plans --path ./team-plans --no-init-git
# 注册已经克隆的 store
openspec store register ./team-plans --id team-plans
openspec list --store team-plans
openspec new change add-shared-theme --store team-plans
```

子命令	中文说明
store setup [id]	创建并注册；非交互需 id 和 --path；可写 --remote；默认 init-git
store register [path]	注册已有目录；--id；--yes 确认创建身份元数据
store list / store ls	列本机注册项
store doctor [id]	只读诊断注册、元数据与 Git 状态
store unregister <id>	取消本机注册，保留仓库文件
store remove <id>	取消注册并删除本地目录；非交互需 --yes，谨慎使用

上述命令支持 --json。注册是本机路径映射，不会自动让同事拥有相同目录。共享靠 Git：同事先克隆，再注册同一逻辑 ID。OpenSpec 不自动 clone、pull 或 push。Beta 的命令、格式与 JSON 可能变化，团队应固定版本。

选择写入位置，与提供参考资料是两件事

```
# 方案 A：本仓库有自己的规格，只引用平台规格
schema: spec-driven
references:
  - platform-contracts

# 方案 B：本仓库只保留配置，规划放外部 store
# store: team-plans
```

references 提供只读能力索引和读取命令，不把规格全文复制进提示，不改变写入根目录。store: 是配置型仓库的默认规划位置，不应与真实本地 specs/changes/ 混用。[R16]

写到哪里：解析顺序

显式 --store → 本地规划根目录 → config-only 仓库的 store: → 本机 defaultStore。存在本地规划目录时，store: 不会覆盖它。写入前检查命令输出 root 信息；无法解析显式 store 时应修正注册，不要悄悄退回本地。

```
openspec doctor --json
openspec context --json
openspec context --code-workspace planning.code-workspace
```

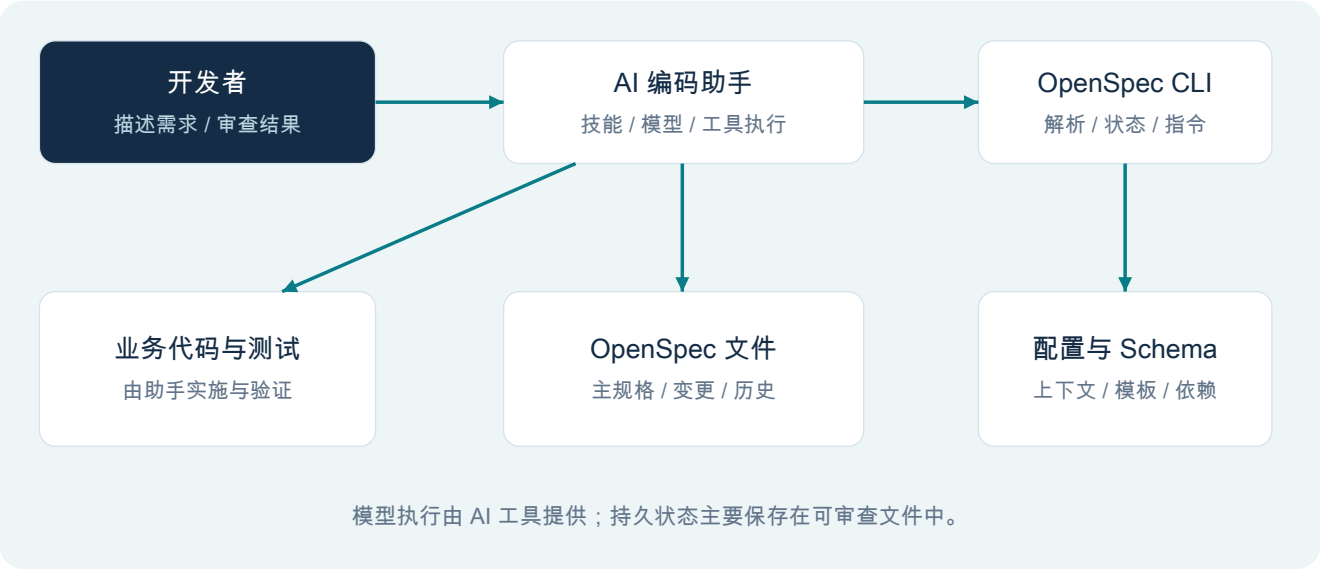
doctor 只诊断关系健康；健康报告有发现也可能退出 0，自动化应读取 status 内容。context 展示根目录与引用仓库的工作集合，--code-workspace 才会写工作区文件，覆盖已有文件需 --force。

Workset：个人常用文件夹组合（Beta）

```
openspec workset create theme-work --member ./team-plans --member ./web-app --tool codex
openspec workset list --json
openspec workset open theme-work
openspec workset remove theme-work
```

create 的第一个 member 是主要目录；open 在编辑器中打开多文件夹，或启动附带这些目录的 CLI 助手会话。--tool 可覆盖已保存工具。remove 只移除本机视图，非交互需 --yes，不删除成员仓库。Workset 不提交、不共享、不根据 references 自动推导。[R17]

人、助手、CLI 与文件如何配合



图为依据官方命令契约整理的逻辑架构，展示职责与数据流，并非官方进程部署图。OpenSpec 不内置一个代替所有助手的模型服务；模型调用和编码执行由你选用的 AI 工具完成。[R6][R12][R13]

一次 propose 的执行链

1. 工具加载生成的技能/命令，知道要按 OpenSpec 流程处理请求。
2. 助手通过 CLI 解析项目或 store 根目录、schema 和产物状态。
3. instructions 将模板、上下文、规则、依赖等输入交给助手。
4. 助手读取已有代码/规格，按依赖写出 proposal、delta、design/tasks。
5. 用户审查；apply 继续读取产物和任务，修改业务代码并验证。
6. sync/archive 把已确认行为合并进主规格，保留历史变更。

想读源码，从这些职责入手

src/cli/index.ts：CLI 命令入口与参数。

src/commands/workflow/：status、instructions、new 等工作流支撑。

schemas/spec-driven/schema.yaml：默认产物依赖、模板与指导。

src/core/：初始化、工具适配、归档和规格相关能力。

先对照一次真实命令输出，再跟到对应实现，避免从整个仓库逐文件阅读。

先定位失败发生在哪一层

现象	优先检查	处理方向
openspec 找不到	PATH 与安装位置	新开终端；which -a / where.exe；确认实际副本
npm 安装后报 Node 错误	node --version	升到 ≥20.19.0，或改独立版本
Bun 安装成功却 env: node 错误	系统 Node 是否可见	补 Node，或改独立版本
macOS 手动下载不能运行	执行权限、来源与隔离属性	chmod；核对可信来源后处理 quarantine
/opsx 指令不识别	工具的实际调用语法与 profile	查看 init 提示；重启工具；运行 update
Codex 找不到 /opsx:propose	是否使用了通用名称	使用 $openspec-propose，检查 .agents/skills/
没有 verify / continue	当前默认是 core	config profile 勾选扩展项，再 update
update 提示最新但缺新功能	当前运行 CLI 版本/安装来源	version；升级正确副本，再刷新项目
JSON 前混入其他文本	调用方式与输出流	检查 stdout/stderr；不要把整段终端文本当 JSON

一个低风险的诊断顺序

```
openspec --version
openspec version
openspec config list
openspec update --dry-run
openspec list
openspec status --all --json
```

只有确认目录、来源和计划后，才实际 update。不要先用 --force 清理问题现场。若没有 OpenSpec 项目，应在目标项目根目录 init；不要误初始化用户主目录或临时错误目录。[R5][R12]

没有计划，apply 被阻塞

查看 status --change <名称> 的 missingDeps，生成缺少的产物，或检查 schema 对条件省略的定义。不要把空文件或空复选框列表当作完成计划。

结构通过以后，还要检查语义

现象	原因与处理
没识别到场景	使用 #### Scenario:；需求使用 ### Requirement:；不要翻译这些结构关键字
需求缺 SHALL/MUST	写清强制行为；将“合理处理”改为明确可观察的结果
MODIFIED 找不到目标	主规格能力路径或需求名称错误；先 list --specs 与 show
修改后丢失旧场景	MODIFIED 写了局部片段；从基线复制完整块后再改
ADDED 与主规格重名	可能是修改需求，重新判断 ADDED/MODIFIED
无 delta 校验失败	行为有变化就补规格；纯重构显式 skip_specs: true
status 显示完成但功能没实现	状态指计划产物；看 tasks 并检查测试与代码
归档无终端报确认失败	确认效果后用 archive <名称> --yes；脚本必须指定目标
删除能力最后一条需求失败	需要 retire_capabilities: true；检查非需求内容的保留要求
已归档却主规格未更新	是否跳过了 specs；检查同步摘要、实际根目录和 delta
store 无法解析	本机未注册、路径移动或指针有误；store list/doctor 检查

安全处理退役能力

REMOVED 删除最后一条需求会移除主 spec.md，必须在变更元数据中明确 retire_capabilities: true。该文件有额外非标准内容时可能拒绝退役，先按错误信息决定内容归属。不要通过 --no-validate 隐藏需要保留的知识。[R10][R12]

回到一致状态

先检查版本管理差异，确认当前主规格、活跃 delta、归档目录和代码各处于什么状态。选择补齐实现、修订 delta 或回退未确认修改，再运行严格校验。对失败的 store 操作检查实际 root，不在另一个仓库修“同名文件”。

> validate 是结构与部分 delta 一致性检查，verify 是助手语义核对，项目测试验证行为。三者互补；任何一项单独通过都不足以证明全部需求已交付。

用最少的规则保住最关键的质量

新手日常清单

1. 先读取现有能力，避免重新发明同名或近义规格。

2. 每个 change 保持单一意图，先审规格再让助手实现。

3. 任务写出“怎么证明完成”，测试与相关实现一起完成。

4. 需求变了就更新文档；换会话时明确变更名，由文件恢复上下文。

5. 归档前检查代码、测试、任务、delta 是否一致，归档后看主规格。

团队建议：CI 的基础命令

```
# CI 环境固定 CLI 版本后运行
npm install -g @fission-ai/openspec@1.15.0
openspec validate --all --strict --no-interactive
openspec validate --archived --no-interactive
# 再运行该项目自己的测试、构建与必要静态检查
```

Node 安装法需要 Node ≥20.19.0；独立二进制 CI 可改用固定发布资产。跨仓库 CI 还需显式克隆并注册所依赖的 store，固定对应提交；仅声明 references 不会下载资料。

逐步推广，避免仪式过重

第一周让一个小功能跑通；第二周统一配置与评审顺序；发现重复需求后再调整能力边界；跨仓库确有需要再试 Stores。不要为标点修正强行编造四份长文档。

上下文与隐私

生成配置只放必要项目事实，不含密钥与真实客户资料。匿名统计默认开启，可用 openspec config set telemetry.enabled false 关闭。OpenSpec 的统计与 AI 服务读取代码的策略是两件事，后者按实际助手及组织配置处理。[R1][R12]

> 最有效的规格能帮助下一位开发者判断“什么必须保持”。持续删掉过时假设，保留当前行为与必要的设计理由。

把复杂需求拆成可独立验收的 Change

保留原进阶篇，并用本页补齐大型任务的拆分方法。以“订单支付”举例：订单创建、模拟支付适配、失败重试、对账查询可以分阶段，但退款不应顺手夹在登录 PR 中。

变更	对外验收	依赖与交付边界
add-order-draft	创建草稿订单且总额一致	不依赖支付成功
add-checkout-payment	对同一请求避免重复支付	依赖订单 ID 与幂等键约定
add-payment-status	查询明确成功、失败、待确认结果	依赖支付状态模型
add-payment-refund	满足退款条件后可追溯退款	单独确认权限与资金规则

可复制提示词：AI 聊天

```
/opsx:explore
请把订单支付需求拆成可独立实现和验收的变更。
为每个变更列 capability 路径、前置接口、验收场景、
数据迁移与回退风险；指出必须先决定的跨团队契约。
不把自然语言依赖清单当作 OpenSpec 自动调度机制。
```

计划与实施规范

同一能力可被多个 change 演进，但同一需求的并行 MODIFIED 需要指定协调人。先稳定接口契约，再允许消费者用模拟实现并行开发；集成测试必须覆盖真实适配。把验收写成可观察行为，把执行顺序写入 tasks，把团队依赖记录在提案/设计或任务管理系统。

避免三个常见误区

超大 change 让审查失效：拆为可交付的行为切片。过小 change 每次重写全部文档：仅写本轮真实差异。误认为 OpenSpec 会自动安排分支、锁定文件或强制审核：这些需 Git、CI 和团队权限配合。

让结构、测试与主规格进入持续检查

以下是 GitHub Actions 的团队配置示例，不是 OpenSpec 自动生成的官方模板。文件放 .github/workflows/quality.yml。Node 22 满足 CLI 与案例运行要求；OpenSpec 固定 v1.15.0，升级时先审阅发行说明。[R12]

```
name: Quality
on:
  pull_request:
  push:
    branches:
      - main
permissions:
  contents: read
jobs:
  verify:
    runs-on: ubuntu-latest
    env:
      OPENSPEC_TELEMETRY: "0"
      OPENSPEC_NO_UPDATE_CHECK: "1"
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
      - name: Install fixed OpenSpec
        run: npm install -g @fission-ai/openspec@1.15.0
      - name: Validate active specs and changes
        run: openspec validate --all --strict --no-interactive
      - name: Validate archived tasks
        run: openspec validate --archived --no-interactive
      - name: Run Todo tests
        run: npm test
```

成功检查与排错

CI 日志必须同时显示结构校验和业务测试成功；任务未完成的归档应失败。此 Todo 没有依赖安装步骤；真实项目有 lockfile 时按项目方式 npm ci 后测试。空项目尚无 change/spec 时，先判断是否应运行 OpenSpec 检查；不要为了让无内容项目通过而虚构需求。

verify 助手工作流不作为这里的 shell 命令；业务构建、安全扫描和部署门禁按项目补充。跨仓库引用需提前克隆并注册到固定提交，doctor 发现问题时还要读取结构化状态，不能只看退出 0。

两轮案例的实际运行结果



图中第一轮任务在新版中保留完成状态，并可设置高优先级和闰年截止日期。日期清空后再次刷新也已验证。配套项目保存的是应用示例，不包含任何真实个人数据。

从源码包运行

解压配套 ZIP，终端进入 v1/，运行 npm test，再运行 npm start。体验升级时停止 v1，进入 v2/ 再 npm start，并保持同一浏览器和 http://127.0.0.1:5173 地址。

已有归档规格用于对照；若要在源码包上练习新变更，先执行 openspec init --tools <你的工具ID>，再从读取主规格开始。

源码阅读顺序

package.json → server.mjs → index.html/style.css → model.mjs → app.mjs → model.test.mjs。共用文件只印一次，两轮均使用同一份；v1/v2 各自模型和完整测试独立列出。跨页代码按页标题顺序拼接，中间不添加额外省略号。

v1/package.json（ 1/1 ）

v2 同样使用这些 scripts 与 engines，仅 version 可改为 2.0.0。文件不含第三方依赖。

```
{
  "name": "openspec-todo-demo",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node server.mjs",
    "test": "node --test"
  },
  "engines": {
    "node": ">=20.19.0"
  }
}
```

v1/server.mjs（ 1/1 ）

完整文件第 1 至 38 行，顺序拼接各页即可。

```
import { createServer } from "node:http";
import { readFile } from "node:fs/promises";

const files = new Map([
    ["/", ["index.html", "text/html"]],
    ["/index.html", ["index.html", "text/html"]],
    ["/app.mjs", ["app.mjs", "text/javascript"]],
    ["/model.mjs", ["model.mjs", "text/javascript"]],
    ["/style.css", ["style.css", "text/css"]]
]);
const port = Number(process.env.PORT || 5173);
const server = createServer(async (req, res) => {
    const path = new URL(req.url, "http://localhost").pathname;
    const entry = files.get(path);
    if (req.method !== "GET" || !entry) {
        res.writeHead(404);
        res.end("Not found");
        return;
    }
    try {
        const body = await readFile(new URL(entry[0], import.meta.url));
        res.writeHead(200, {
            "Content-Type": `${entry[1]}; charset=utf-8`,
            "Cache-Control": "no-store"
        });
        res.end(body);
    } catch {
        res.writeHead(500);
        res.end("Unable to read local file");
    }
});
server.on("error", (error) => {
    console.error(error.message);
    process.exitCode = 1;
});
server.listen(port, "127.0.0.1", () => {
    console.log(`Todo: http://127.0.0.1:${port}`);
});
```

v1/index.html（ 1/2 ）

完整文件第 1 至 23 行，顺序拼接各页即可。

```
<!doctype html>
<html lang="zh-CN">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>OpenSpec Todo 实战</title>
  <link rel="stylesheet" href="./style.css">
  <script type="module" src="./app.mjs"></script>
</head>
<body>
  <main>
    <p class="eyebrow">OPENSPEC / LEARN BY BUILDING</p>
    <h1>我的待办事项</h1>
    <form id="todo-form" novalidate>
      <label for="title">任务标题</label>
      <input id="title" name="title" maxlength="120"
        placeholder="例如：完成 OpenSpec 实战">
      <div id="extra-fields" hidden>
        <label>优先级
          <select name="priority">
            <option value="low">低</option>
            <option value="medium" selected>中</option>
            <option value="high">高</option>
```

v1/index.html（2/2）

完整文件第 24 至 46 行，顺序拼接各页即可。

```
                </select>
            </label>
            <label>截止日期
                <input name="dueDate" type="date"
                    min="0001-01-01" max="9999-12-31">
            </label>
        </div>
        <button type="submit">添加任务</button>
    </form>
    <p id="message" role="status" aria-live="polite"></p>
    <p id="empty">还没有任务，添加第一项吧。</p>
    <ul id="todos"></ul>
  </main>
  <template id="fields">
    <select>
      <option value="low">低</option>
      <option value="medium">中</option>
      <option value="high">高</option>
    </select>
    <input type="date" min="0001-01-01" max="9999-12-31">
  </template>
</body>
</html>
```

v1/style.css（ 1/1 ）

完整文件第 1 至 42 行，顺序拼接各页即可。

```
* { box-sizing: border-box; }
body {
  margin: 0;
  background: #edf3f7;
  color: #17324d;
  font: 16px/1.6 system-ui, sans-serif;
}
main {
  max-width: 840px;
  margin: 40px auto;
  padding: 32px;
  background: white;
  border-radius: 16px;
}
.eyebrow { color: #087d88; font-size: 12px; letter-spacing: 2px; }
input, select, button {
  font: inherit;
  padding: 8px 12px;
  border: 1px solid #b7cbd6;
  border-radius: 6px;
}
#title { width: 100%; margin: 8px 0 12px; }
#extra-fields { margin-bottom: 12px; }
button { cursor: pointer; background: #edf5f7; color: #17324d; }
button[type="submit"] { background: #087d88; color: white; }
#message { min-height: 26px; color: #825018; }
ul { padding: 0; }
li {
  display: flex;
  gap: 8px;
  align-items: center;
  flex-wrap: wrap;
  padding: 12px 0;
  border-bottom: 1px solid #dce7ee;
}
li label { flex: 1; overflow-wrap: anywhere; min-width: 140px; }
li input[type="checkbox"] { margin-right: 10px; }
.done { text-decoration: line-through; color: #687b8b; }
:focus-visible { outline: 3px solid #e5a33b; outline-offset: 3px; }
@media (max-width: 600px) {
  main { margin: 12px; padding: 20px; }
}
```

v1/app.mjs（ 1/3 ）

两版共用。extended=false 时不启用预留扩展控件，第一轮对外功能仍为基础 Todo。

```
import {
  extended, createTodo, updateTodo, removeTodo, readTodos, persistTodos
} from "./model.mjs";

const form = document.querySelector("#todo-form");
const list = document.querySelector("#todos");
const message = document.querySelector("#message");
let storage;
try {
  storage = window.localStorage;
} catch {
  storage = null;
}
let todos = readTodos(storage);
document.querySelector("#extra-fields").hidden = !extended;

function commit() {
  const saved = persistTodos(storage, todos);
  message.textContent = saved ? "已保存" : "无法保存，刷新可能丢失修改";
  render();
}

function change(id, patch) {
  try {
    todos = todos.map((todo) => {
      return todo.id === id ? updateTodo(todo, patch) : todo;
    });
    commit();
  } catch (error) {
    message.textContent = error.message;
    render();
  }
}
```

v1/app.mjs（2/3）

完整文件第 34 至 66 行，顺序拼接各页即可。

```
function render() {
  list.replaceChildren();
  document.querySelector("#empty").hidden = todos.length !== 0;
  for (const todo of todos) {
    const row = document.createElement("li");
    const label = document.createElement("label");
    const check = document.createElement("input");
    check.type = "checkbox";
    check.checked = todo.completed;
    check.addEventListener("change", () => {
      change(todo.id, { completed: check.checked });
    });
    const title = document.createElement("span");
    title.textContent = todo.title;
    title.className = todo.completed ? "done" : "";
    label.append(check, title);
    row.append(label);
    if (extended) {
      const template = document.querySelector("#fields");
      const fields = template.content.cloneNode(true);
      const priority = fields.querySelector("select");
      const dueDate = fields.querySelector("input");
      priority.value = todo.priority;
      dueDate.value = todo.dueDate;
      priority.setAttribute("aria-label", `${todo.title}的优先级`);
      dueDate.setAttribute("aria-label", `${todo.title}的截止日期`);
      priority.addEventListener("change", () => {
        change(todo.id, { priority: priority.value });
      });
      dueDate.addEventListener("change", () => {
        change(todo.id, { dueDate: dueDate.value });
      });
```

v1/app.mjs（3/3）

完整文件第 67 至 97 行，顺序拼接各页即可。

```
            row.append(fields);
        }
        const remove = document.createElement("button");
        remove.textContent = "删除";
        remove.setAttribute("aria-label", `删除${todo.title}`);
        remove.addEventListener("click", () => {
            todos = removeTodo(todos, todo.id);
            commit();
        });
        row.append(remove);
        list.append(row);
    }
}

form.addEventListener("submit", (event) => {
    event.preventDefault();
    try {
        const fields = {
            priority: form.elements.priority.value,
            dueDate: form.elements.dueDate.value
        };
        const todo = createTodo(form.elements.title.value, fields);
        todos.push(todo);
        form.reset();
        commit();
        form.elements.title.focus();
    } catch (error) {
        message.textContent = error.message;
    }
});
render();
```

v1/model.mjs（ 1/2 ）

完整文件第 1 至 31 行，顺序拼接各页即可。

```
export const extended = false;
export const storageKey = "openspec-todo-demo";

export function cleanTitle(value) {
    const title = String(value ?? "").trim();
    if (!title || title.length > 120) {
        throw new Error("标题须为 1 至 120 个字符");
    }
    return title;
}

export function createTodo(title, fields = {}) {
    return {
        id: crypto.randomUUID(),
        title: cleanTitle(title),
        completed: false
    };
}

export function updateTodo(todo, patch) {
    if (typeof patch.completed !== "boolean") {
        throw new Error("完成状态必须是布尔值");
    }
    return { ...todo, completed: patch.completed };
}

export function removeTodo(todos, id) {
    return todos.filter((todo) => todo.id !== id);
}

export function normalizeTodos(value) {
```

v1/model.mjs（2/2）

完整文件第 32 至 62 行，顺序拼接各页即可。

```
    if (!Array.isArray(value)) {
      return [];
    }
    return value.filter((item) => {
      return item && typeof item.id === "string" &&
        typeof item.title === "string" && item.title.trim() &&
        item.title.length <= 120 &&
        typeof item.completed === "boolean";
    }).map((item) => ({
      id: item.id,
      title: item.title,
      completed: item.completed
    }));
}

export function readTodos(storage) {
    try {
      return normalizeTodos(JSON.parse(storage.getItem(storageKey)));
    } catch {
      return [];
    }
}

export function persistTodos(storage, todos) {
    try {
      storage.setItem(storageKey, JSON.stringify(todos));
      return true;
    } catch {
      return false;
    }
}
```

v1/model.test.mjs（ 1/1 ）

完整文件第 1 至 43 行，顺序拼接各页即可。

```
import test from "node:test";
import assert from "node:assert/strict";
import {
  createTodo, updateTodo, removeTodo, readTodos, persistTodos, normalizeTodos
} from "./model.mjs";

test("新增并去除首尾空格", () => {
  const todo = createTodo("  学习 OpenSpec  ");
  assert.equal(todo.title, "学习 OpenSpec");
  assert.equal(todo.completed, false);
  assert.ok(todo.id);
});
test("拒绝空白和超长标题", () => {
  assert.throws(() => createTodo("   "));
  assert.throws(() => createTodo("a".repeat(121)));
});
test("切换完成状态且不改变原任务", () => {
  const todo = createTodo("测试");
  assert.equal(updateTodo(todo, { completed: true }).completed, true);
  assert.equal(todo.completed, false);
});
test("删除时只移除选定任务", () => {
  const a = createTodo("任务 A");
  const b = createTodo("任务 B");
  assert.deepEqual(removeTodo([a, b], a.id), [b]);
});
test("存储往返和非法内容降级", () => {
  let value = "";
  const storage = {
    setItem(key, data) { value = data; },
    getItem() { return value; }
  };
  const todos = [createTodo("保留我")];
  assert.equal(persistTodos(storage, todos), true);
  assert.deepEqual(readTodos(storage), todos);
  assert.deepEqual(readTodos({ getItem() { return "invalid"; } }), []);
  assert.deepEqual(normalizeTodos([null, {}]), []);
});
test("存储不可用时可继续内存操作", () => {
  assert.deepEqual(readTodos(null), []);
  assert.equal(persistTodos(null, []), false);
  assert.equal(createTodo("仍能添加").title, "仍能添加");
});
```

v2/model.mjs（ 1/3 ）

完整文件第 1 至 32 行，顺序拼接各页即可。

```
export const extended = true;
export const storageKey = "openspec-todo-demo";

export function cleanTitle(value) {
  const title = String(value ?? "").trim();
  if (!title || title.length > 120) {
    throw new Error("标题须为 1 至 120 个字符");
  }
  return title;
}

export function cleanFields(fields) {
  const priority = fields.priority ?? "medium";
  const dueDate = fields.dueDate ?? "";
  if (!["low", "medium", "high"].includes(priority)) {
    throw new Error("优先级必须为 low、medium 或 high");
  }
  if (dueDate !== "") {
    const date = new Date(`${dueDate}T00:00:00.000Z`);
    if (!/^\d{4}-\d{2}-\d{2}$/.test(dueDate) ||
      dueDate < "0001-01-01" ||
      Number.isNaN(date.getTime()) ||
      date.toISOString().slice(0, 10) !== dueDate) {
      throw new Error("截止日期必须是有效的 YYYY-MM-DD");
    }
  }
  return { priority, dueDate };
}

export function createTodo(title, fields = {}) {
  return {
    id: crypto.randomUUID(),
```

v2/model.mjs（2/3）

完整文件第 33 至 64 行，顺序拼接各页即可。

```
        title: cleanTitle(title),
        completed: false,
        ...cleanFields(fields)
    };
}

export function updateTodo(todo, patch) {
    const next = { ...todo, ...patch };
    if (typeof next.completed !== "boolean") {
        throw new Error("完成状态必须是布尔值");
    }
    return { ...next, ...cleanFields(next) };
}

function safeDate(value) {
    try {
        return cleanFields({ dueDate: value }).dueDate;
    } catch {
        return "";
    }
}

export function removeTodo(todos, id) {
    return todos.filter((todo) => todo.id !== id);
}

export function normalizeTodos(value) {
    if (!Array.isArray(value)) {
        return [];
    }
    return value.filter((item) => {
        return item && typeof item.id === "string" &&
```

v2/model.mjs（ 3/3 ）

完整文件第 65 至 95 行，顺序拼接各页即可。

```
            typeof item.title === "string" && item.title.trim() &&
            item.title.length <= 120 &&
            typeof item.completed === "boolean";
    }).map((item) => ({
        id: item.id,
        title: item.title,
        completed: item.completed,
        ...cleanFields({
            priority: ["low", "medium", "high"].includes(item.priority)
                ? item.priority : "medium",
            dueDate: safeDate(item.dueDate)
        })
    }));
}

export function readTodos(storage) {
    try {
        return normalizeTodos(JSON.parse(storage.getItem(storageKey)));
    } catch {
        return [];
    }
}

export function persistTodos(storage, todos) {
    try {
        storage.setItem(storageKey, JSON.stringify(todos));
        return true;
    } catch {
        return false;
    }
}
```

v2/model.test.mjs（ 1/2 ）

完整文件第 1 至 40 行，顺序拼接各页即可。

```
import test from "node:test";
import assert from "node:assert/strict";
import {
  createTodo, updateTodo, removeTodo, readTodos, persistTodos, normalizeTodos
} from "./model.mjs";

test("新增并去除首尾空格", () => {
  const todo = createTodo("  学习 OpenSpec  ");
  assert.equal(todo.title, "学习 OpenSpec");
  assert.equal(todo.completed, false);
  assert.ok(todo.id);
});
test("拒绝空白和超长标题", () => {
  assert.throws(() => createTodo("   "));
  assert.throws(() => createTodo("a".repeat(121)));
});
test("切换完成状态且不改变原任务", () => {
  const todo = createTodo("测试");
  assert.equal(updateTodo(todo, { completed: true }).completed, true);
  assert.equal(todo.completed, false);
});
test("删除时只移除选定任务", () => {
  const a = createTodo("任务 A");
  const b = createTodo("任务 B");
  assert.deepEqual(removeTodo([a, b], a.id), [b]);
});
test("存储往返和非法内容降级", () => {
  let value = "";
  const storage = {
    setItem(key, data) { value = data; },
    getItem() { return value; }
  };
  const todos = [createTodo("保留我")];
  assert.equal(persistTodos(storage, todos), true);
  assert.deepEqual(readTodos(storage), todos);
  assert.deepEqual(readTodos({ getItem() { return "invalid"; } }), []);
  assert.deepEqual(normalizeTodos([null, {}]), []);
});
test("存储不可用时可继续内存操作", () => {
  assert.deepEqual(readTodos(null), []);
```

v2/model.test.mjs（ 2/2 ）

完整文件第 41 至 80 行，顺序拼接各页即可。

```
    assert.equal(persistTodos(null, []), false);
    assert.equal(createTodo("仍能添加").title, "仍能添加");
});

test("旧数据默认中优先级且无截止日期", () => {
    const old = [{ id: "old", title: "原任务", completed: true }];
    const loaded = normalizeTodos(old);
    assert.deepEqual(loaded[0], {
        ...old[0], priority: "medium", dueDate: ""
    });
});
test("创建和修改优先级日期时保留旧字段", () => {
    const todo = createTodo("任务", {
        priority: "high", dueDate: "2028-02-29"
    });
    const updated = updateTodo(todo, { priority: "low", dueDate: "" });
    assert.equal(updated.id, todo.id);
    assert.equal(updated.title, todo.title);
    assert.equal(updated.priority, "low");
    assert.equal(updated.dueDate, "");
});
test("校验枚举和闰年日期", () => {
    assert.throws(() => createTodo("任务", { priority: "urgent" }));
    assert.throws(() => createTodo("任务", { dueDate: "2027-02-29" }));
    assert.throws(() => createTodo("任务", { dueDate: "2028-2-9" }));
    assert.equal(createTodo("任务", { dueDate: "2028-02-29" })
        .dueDate, "2028-02-29");
});
test("再次读取迁移后的数据保持不变", () => {
    const old = [{ id: "old", title: "原任务", completed: false }];
    const once = normalizeTodos(old);
    assert.deepEqual(normalizeTodos(once), once);
});
test("非法扩展字段降级且不删除任务", () => {
    const old = [{ id: "old", title: "任务", completed: false,
        priority: "bad", dueDate: "2027-02-29" }];
    assert.equal(normalizeTodos(old).length, 1);
    assert.equal(normalizeTodos(old)[0].priority, "medium");
    assert.equal(normalizeTodos(old)[0].dueDate, "");
});
```

继续阅读与核对

本书基于 2026-10-28 再次核对的官方站点、官方 GitHub 仓库文档与 v1.15.0 本机帮助。官网与 main 会继续更新，示例输出和次要参数可能变化；Beta 内容尤其需要以所用版本为准。

[R1] 项目主页与 README

https://github.com/Fission-AI/OpenSpec

[R2] v1.15.0 发布说明

https://github.com/Fission-AI/OpenSpec/releases/tag/v1.15.0

[R3] 多平台安装、升级与卸载

https://openspec.dev/docs/installation

[R4] 官方快速入门

https://openspec.dev/docs/quickstart

[R5] 项目初始化

https://openspec.dev/docs/setup

[R6] 十二个助手工作流的行为契约

https://openspec.dev/docs/skills

[R7] Profiles 与 delivery

https://openspec.dev/docs/profiles

[R8] 支持工具、安装路径与调用方式

https://openspec.dev/docs/supported-tools

[R9] 默认 schema 与模板

https://github.com/Fission-AI/OpenSpec/tree/main/schemas/spec-driven

[R10] 规格写法与 delta 规则

https://github.com/Fission-AI/OpenSpec/blob/main/docs/writing-specs.md

[R11] 实战配方与纯重构示例

https://github.com/Fission-AI/OpenSpec/blob/main/docs/examples.md

核对方法

命令组与关键选项对照本机 v1.15.0 --help；Todo 两轮 delta、主规格及归档任务均已校验通过；两次 CLI 归档实际成功；两版本合计 17 项模型测试及关键浏览器路径通过。原深色模式的 3 项函数测试亦保留。本文不声称已在所有操作系统、全部 AI 助手或真实团队部署中逐项运行。

把这份指南当作入口

[R12] 完整 CLI 参考

https://openspec.dev/docs/cli

[R13] 项目上下文、规则与操作建议

https://openspec.dev/docs/project-config

[R14] 自定义 Schema

https://openspec.dev/docs/customize-schemas

[R15] 团队工作流

https://github.com/Fission-AI/OpenSpec/blob/main/docs/team-workflow.md

[R16] Stores（Beta）

https://openspec.dev/docs/stores

[R17] Worksets（Beta）

https://openspec.dev/docs/worksets

[R18] 上游许可

https://github.com/Fission-AI/OpenSpec/blob/main/LICENSE

英文	本书用词	记忆提示
Source of truth	事实来源 / 当前规格	现在系统承诺什么
Delta	增量规格	这次要改什么
Proposal	提案	为什么改、改多大
Design	设计	如何实现与为何这样选
Tasks	任务	具体做什么、怎样验证
Sync	同步规格	把增量合并进主规格
Archive	归档	保存完成变更的历史
Store	规格仓库	跨代码仓库共享规划
Workset	个人工作集合	重开常用的一组目录

许可与归属

OpenSpec 上游代码与文档采用 MIT License，Copyright (c) 2024 OpenSpec Contributors。本书为独立中文编写与教学整理，引用的名称、命令及文档结构归其原作者；下页保留上游 MIT 许可全文。本书中的流程图为依据公开契约绘制的教学图，实战与团队建议已在正文标明。

上游文档许可

以下保留 OpenSpec 官方 LICENSE 的英文全文。它适用于上游软件及其相关文档的使用与再分发。[R18]

MIT License

Copyright (c) 2024 OpenSpec Contributors

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

最后一次官方资料核对：2026-10-28。建议在升级前运行 openspec version、阅读对应 Release Notes，并先用 openspec update --dry-run 检查项目集成变化。